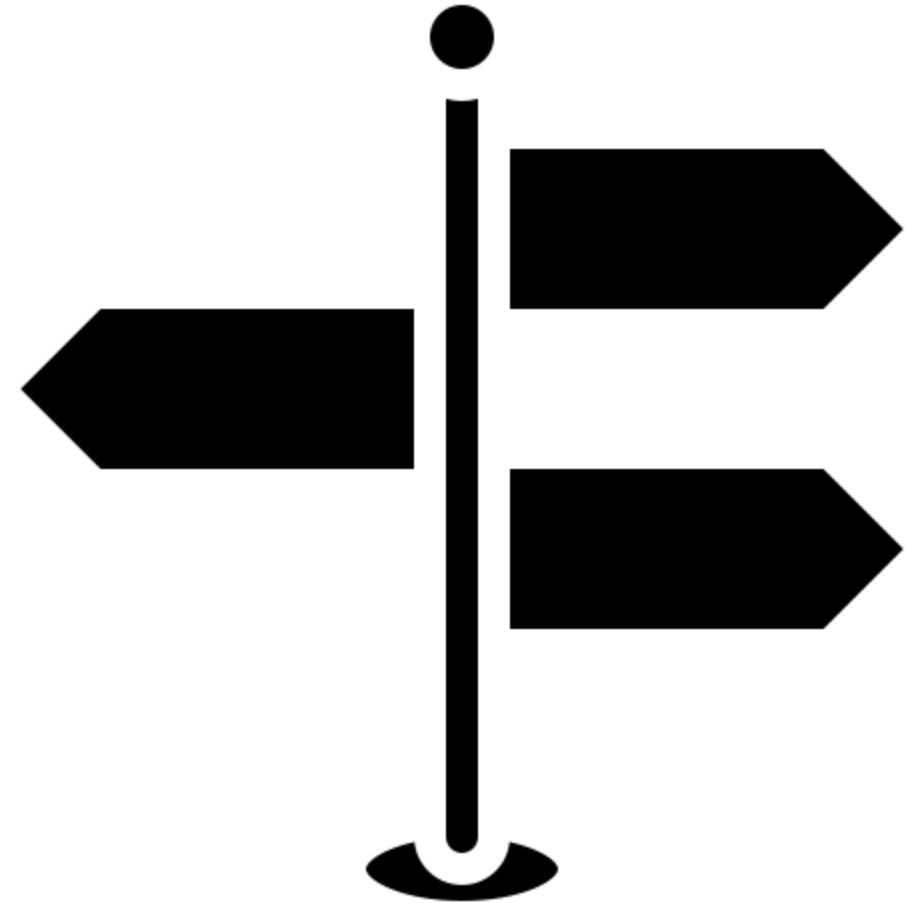


Public Cloud Services Databases and Storage



Learning Objectives

- Students understand the difference between blob and file storage.
- Students can set up blob and file storage in the cloud.
- Students understand the difference between databases and the purposes of blob and file storage.



Agenda

- Storage Concepts
- File Storage
- Block Storage
- Objekt / Blob Storage

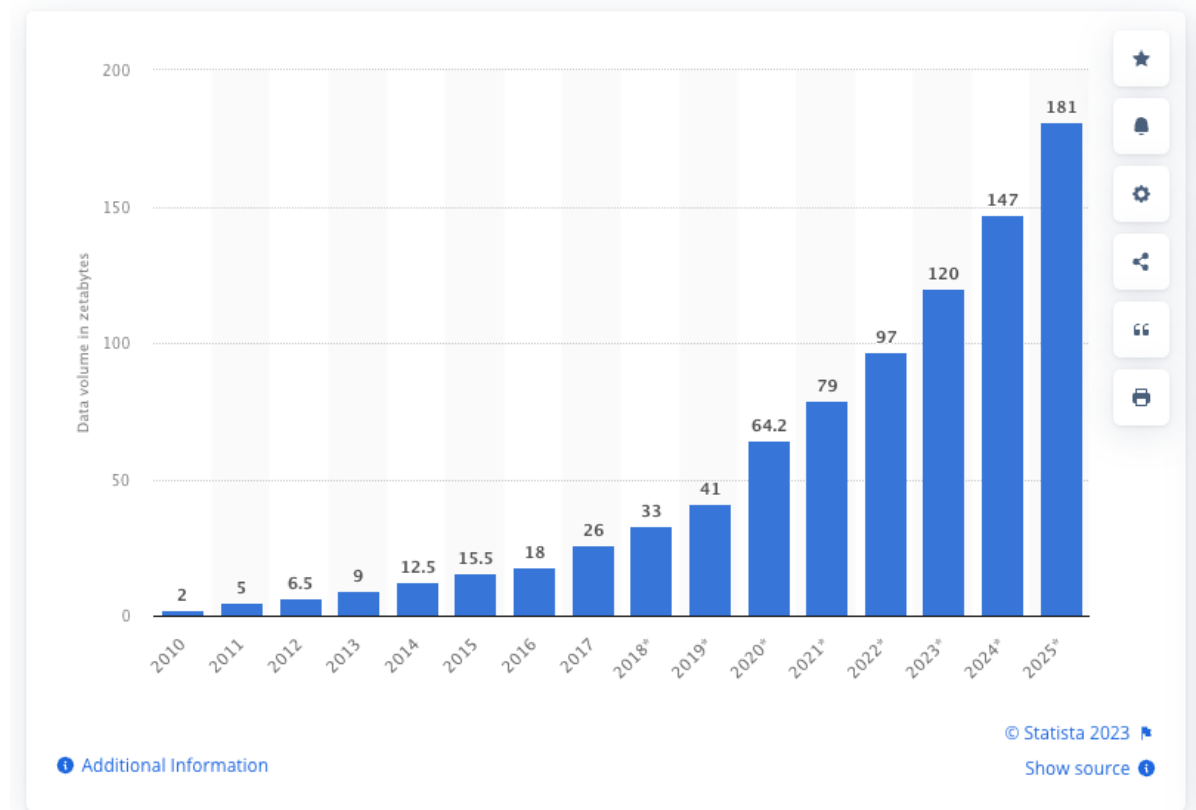
Motivation

1. 262 Pages = 1MB
2. 536 Books = 1GB
3. 549755 Books = 1TB
4. 192 Library of Congress = 1PB

– → Non-linear increase in storage volume per year

– Source:

https://www.bookpedia.de/buecher/Wieviel_Information_gibt_es_in_der_Welt



Storage Concepts

File Storage

- Access is provided, for example, via a POSIX-compatible file system over the network.
- Files can be written partially.
- Network protocols such as NFS allow multiple clients to access the same files.

Block Storage

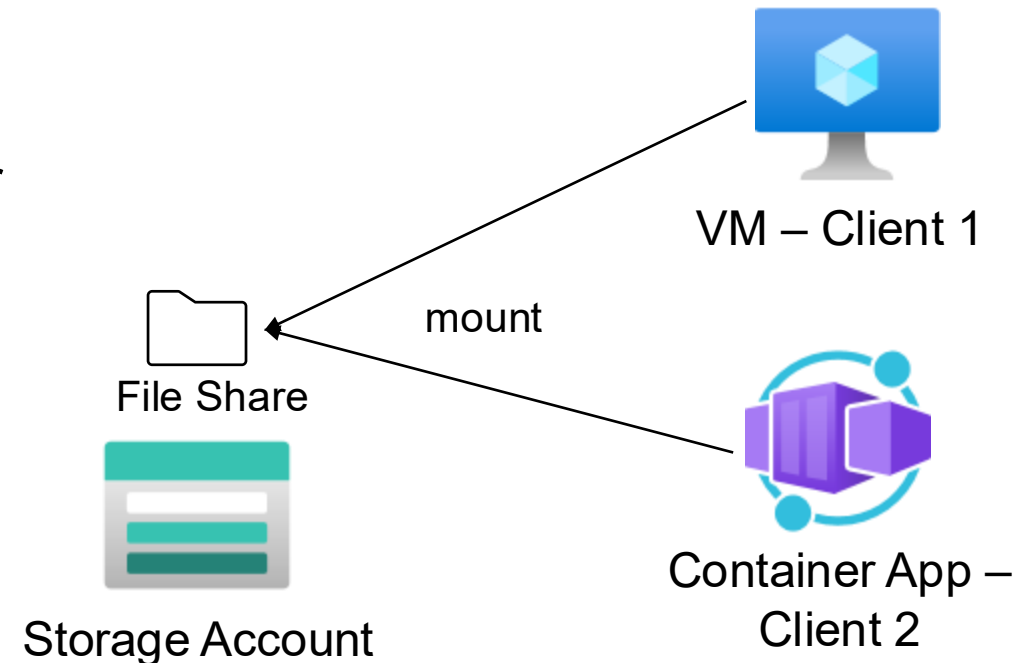
- Accesses are made directly to the device (e.g., disk) attached to a VM.
- Writing is done in blocks, e.g., 512 bytes.
- Mostly used in combination with a file system (such as ext4 or similar).

Blob/Object Storage

- Access always takes place via REST API.
- Objects (binary files, structured text files -> files) are written en bloc.
- Specific extensions are append blobs.

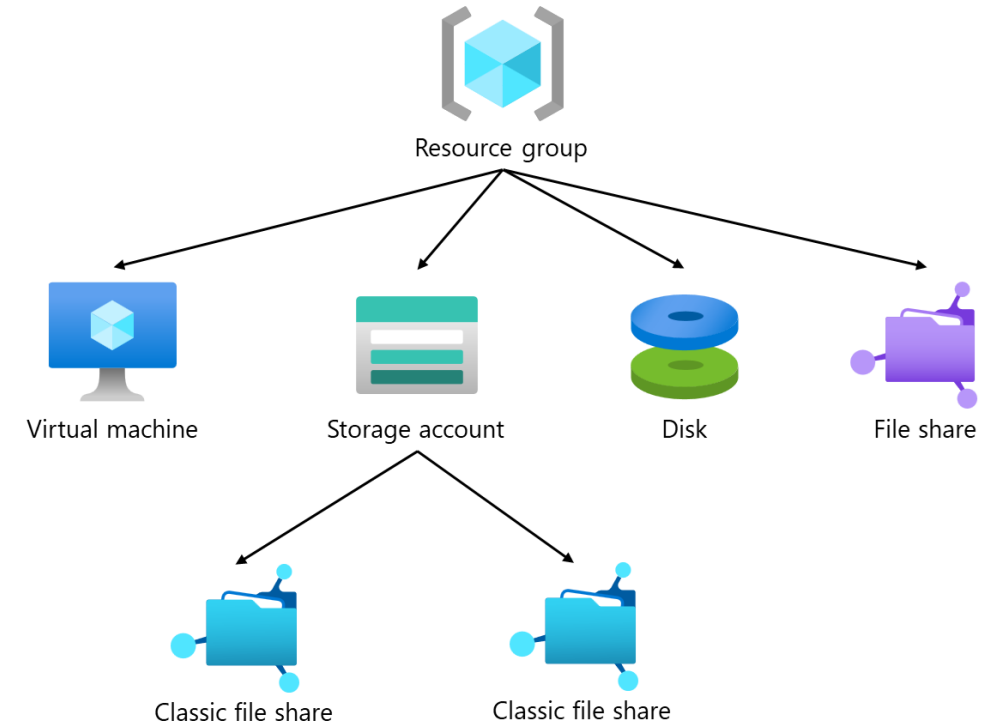
File Storage

- NFS/SMB-compatible storage solution
- Supports various storage types such as SSD or HDD
- Mounting via PowerShell or Bash script
- Use cases
 - POSIX-compatible file system required
 - Multiple clients must be able to work on the same file system
 - Shared file storage



File Storage

- Transition process from “storage account file shares” to “file shares” (service renaming)
- Decoupling of storage accounts
- File shares are still in preview
- Additional premium offerings: Netapp Files



```
az storage share create --account-name MyAccount --name MyFileShare
```

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-planning>

File Storage - Pricing

- Depending on SKU, redundancy, region, storage amount, and IOPS
- Additionally, depending on usage, snapshots and soft delete

Redundancy:

LRS

Region:

Switzerland North

Currency:

Switzerland – Franc (chf) CHF

Display pricing by:

Month

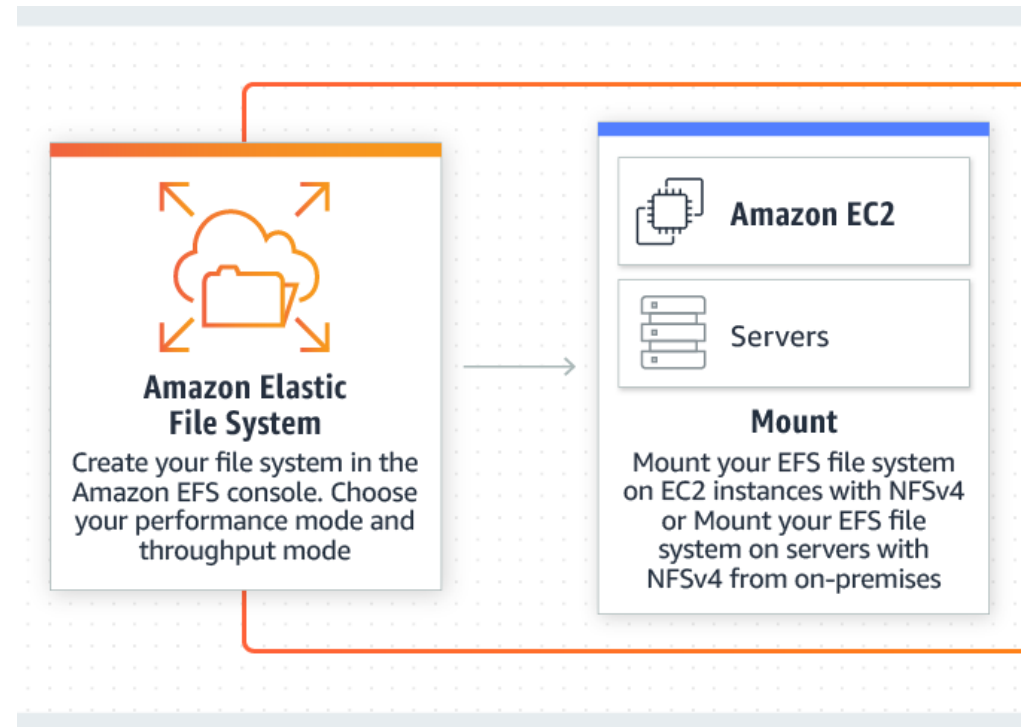
1 USD = 0.804 CHF

	SSD	HDD
Quantity	Price	Price
Provisioned storage The maximum storage size of the file share, specified in GiB. You are charged for the storage you've provision regardless of what you use. Provisioned storage can be changed after share creation.	CHF 0.1151 per provisioned GiB per month	CHF 0.0083 per provisioned GiB per month
Provisioned IOPS The maximum IOPS, or transactions per second, of the file share. You are charged for the IOPS you've provisioned regardless of how many IOs you use. Provisioned IOPS can be changed after share creation.	CHF 0.0312 per provisioned IO per sec per month <i>First 3000 IOPS at no additional cost</i>	CHF 0.0464 per provisioned IO per sec per month
Provisioned throughput The maximum amount of data that can be transferred per second of the file share. You are charged for the throughput you've provisioned regardless of what you use. Provisioned throughput can be changed after share creation.	CHF 0.0452 per provisioned MiB per sec per month <i>First 100 MiB / sec at no additional cost</i>	CHF 0.0687 per provisioned MiB per sec per month

<https://azure.microsoft.com/de-de/pricing/details/storage/files/>

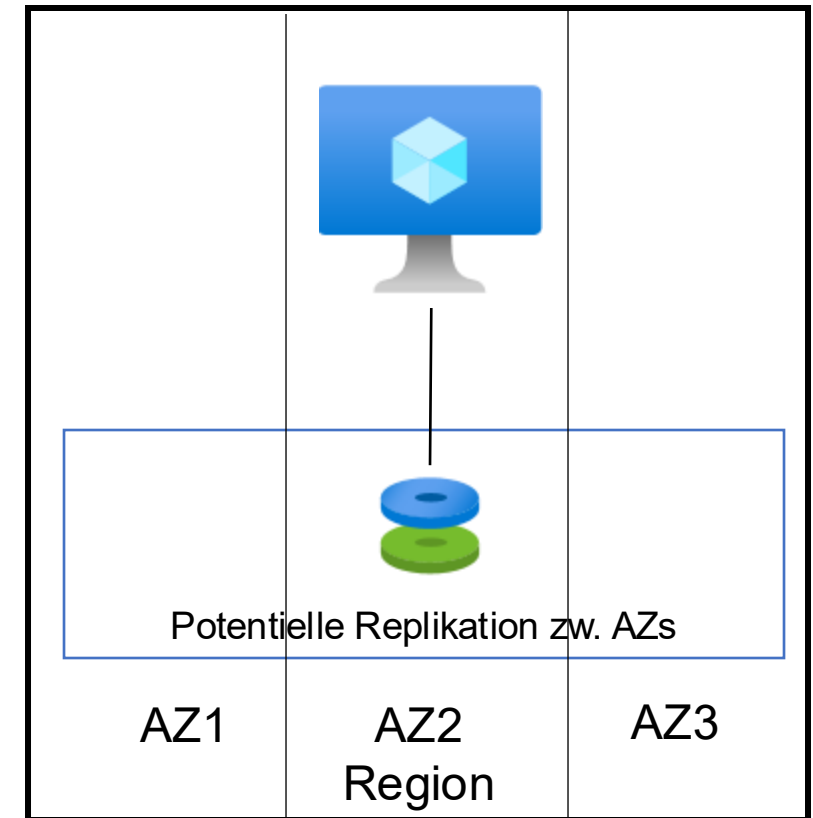
File Storage – AWS Elastic File System (EFS)

- File storage product from AWS, comparable to Azure Files
- Available as NFS, SMB, or Linux storage
- Supports various storage classes (SSD, hybrid, SATA disk)
- Additional specialized services: Amazon FSx for NetApp ONTAP or Amazon FSx for Windows File Server



Block Storage

- Native storage solution for virtual machines in the form of disks
- Performance classes such as SSD, HDD, or Ultra
- Features
 - Snapshots
 - Replication (LRS, ZRS)
 - Encryption
 - Base images



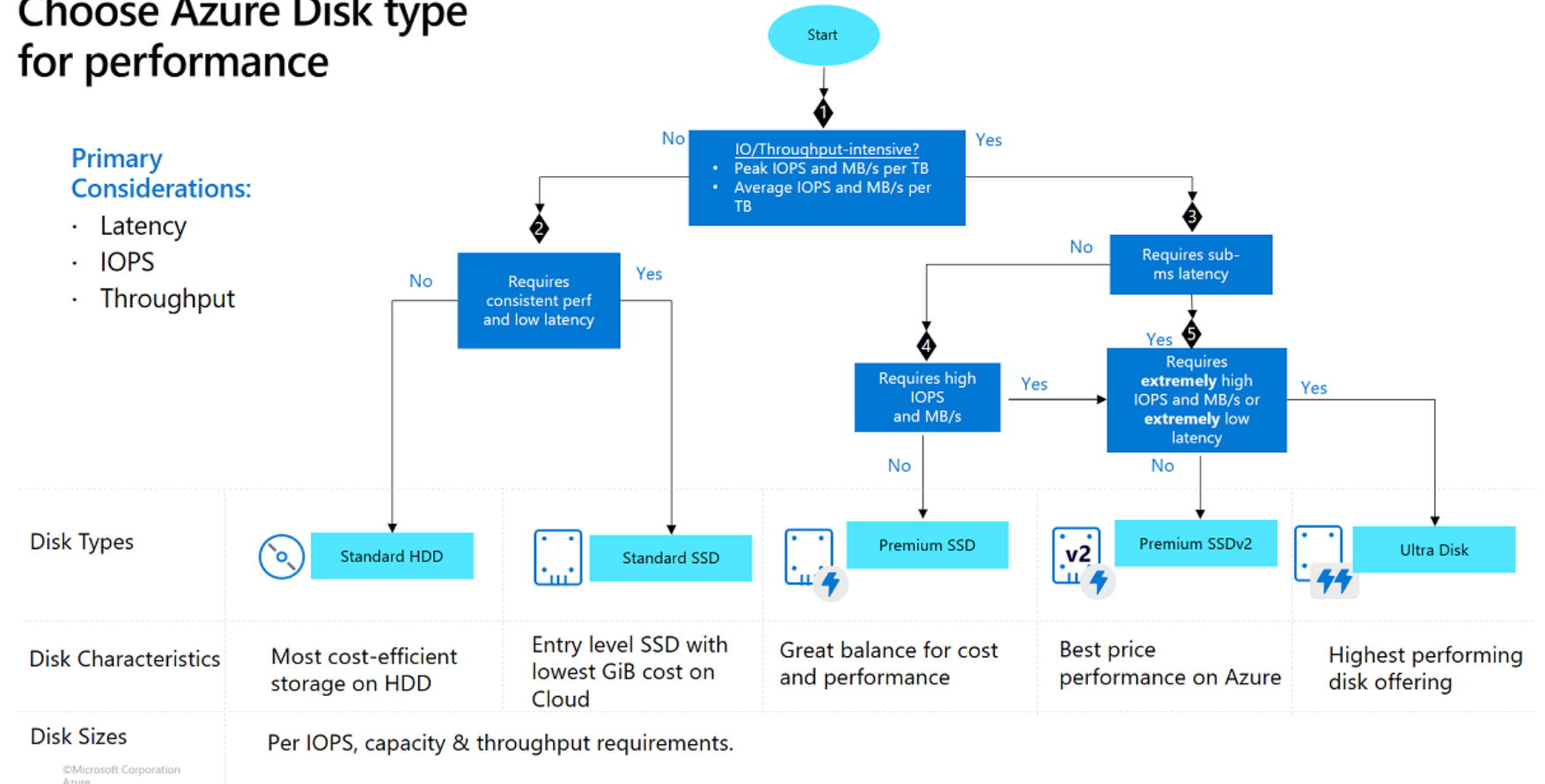
Block Storage

- Selecting the disk type based on performance requirements
- Note: Reserved capacity and IOPS have a significant impact on costs

Choose Azure Disk type for performance

Primary Considerations:

- Latency
- IOPS
- Throughput



<https://learn.microsoft.com/en-us/azure/virtual-machines/disks-types>

Azure Managed Disk

- Azure Disks have a lifecycle that is independent of virtual machines
- Azure Disks can be created from
 - Scratch (empty disk)
 - Snapshots of other disks
 - Base images (e.g., Ubuntu)

```
az disk create -g MyResourceGroup -n MyDisk --size-gb 10
```

```
az vm disk attach \  
  -g myResourceGroup \  
  --vm-name myVM \  
  --name myDataDisk \  
  [ --new \  
    --size-gb 50
```

<https://learn.microsoft.com/en-us/cli/azure/disk?view=azure-cli-latest#az-disk-create>

Azure Managed Disk

- Snapshots are point-in-time backups of disks.
- Snapshots can be created incrementally.
- Snapshots have their own lifecycle and are not dependent on the source (e.g., a disk).
- Billing is based on the storage space actually used.

Create a snapshot:

```
az snapshot create -g MyResourceGroup -n MySnapshot2 --source MyDisk
```

Create a disk from a snapshot:

```
az disk create -g MyResourceGroup -n MyDisk2 --source MyDisk
```

Snapshots are crash consistent!

<https://learn.microsoft.com/en-us/cli/azure/snapshot?view=azure-cli-latest#az-snapshot-create>
<https://learn.microsoft.com/en-us/azure/virtual-machines/managed-disks-overview#snapshots>

Azure Managed Disk

- Azure Storage Services (whether disk, blob, DBs, etc.) are always encrypted at rest.
- The encryption key is platform-managed by default.
- Advanced encryption options with Customer Managed Key (CMK) are supported.

Storage Server Side Encryption

Standard encryption method from storage. Can be configured with CMK.

Encryption at Host

Encryption directly on the host before the blocks are transferred to storage (end-to-end).

Azure Disk Encryption

Encryption of the OS and data directly on the disk (via BitLocker / DM-Crypt).

Confidential Disk Encryption

Encryption using TPM directly in the VM. OS-only supported.

<https://learn.microsoft.com/en-us/azure/storage/common/storage-service-encryption>

<https://learn.microsoft.com/en-us/azure/virtual-machines/disk-encryption-overview>

Block Storage – AWS Elastic Block Store (EBS)

- Equivalent of AWS to Azure Disks
- AWS EBS Volumes = Azure Disks
- EBS Volume types comparable to Azure Disk SKUs

AWS EBS volume type	Azure Managed disk	Use	Can this managed disk be used as an OS Disk
gp2/gp3	Standard SSD	Web servers and lightly used application servers or dev/test environments	Yes
gp2	Premium SSD	Production and performance-sensitive workloads	Yes
gp3	Premium SSD v2	Performance-sensitive workloads or workloads that require high IOPS and low latency	No
io2	Ultra Disk Storage	IO-intensive workloads, performance-demanding databases, and very high transaction workloads that demand high throughput and IOPS	No

On both platforms: The VM SKU must match the block storage SKU. For example, small VMs cannot take advantage of the performance of Ultra Disks!

<https://learn.microsoft.com/en-us/azure/architecture/aws-professional/storage>

Storage Products – Blob / Object Storage



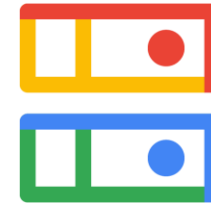
AWS S3

- Buckets as data containers for objects
- Buckets are regional and can be replicated
- Access via globally unique URL



Azure Blob Storage

- Storage Account -> Container -> Blobs
- Storage accounts specify the region (can be replicated)
- Access via globally unique URL



GCP Cloud Storage

- Buckets as data containers for objects
- Buckets are regional and can be replicated
- Access via globally unique bucket names. Base URL is stable.

There are similar features, but the APIs are not compatible!

Object / Blob Storage – Use Cases

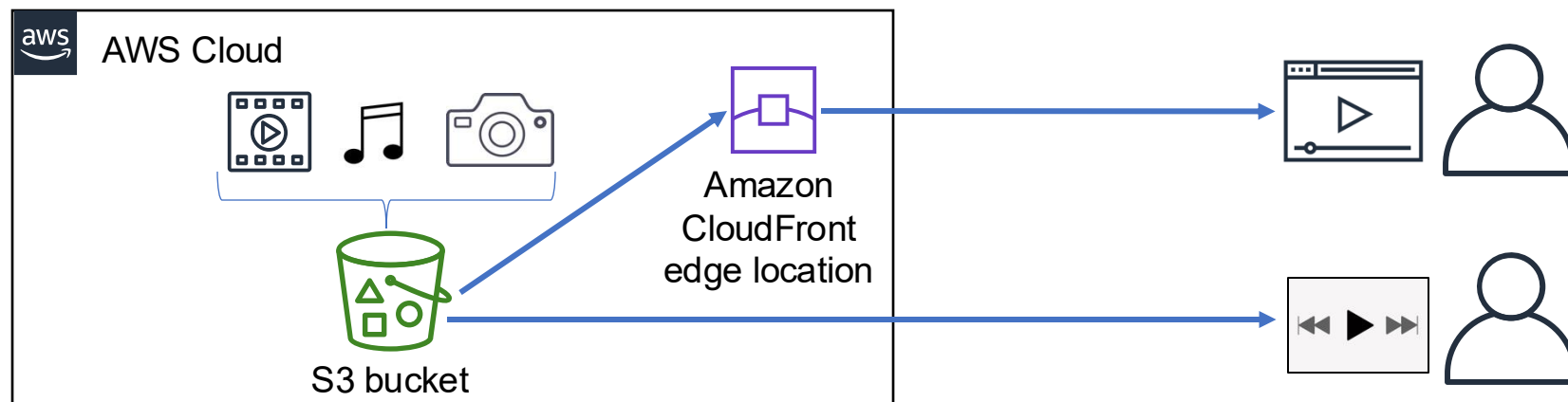
A redundant, scalable, and highly available infrastructure that enables the uploading and downloading of videos, photos, or music.



`https://<bucket-name>.s3.amazonaws.com`

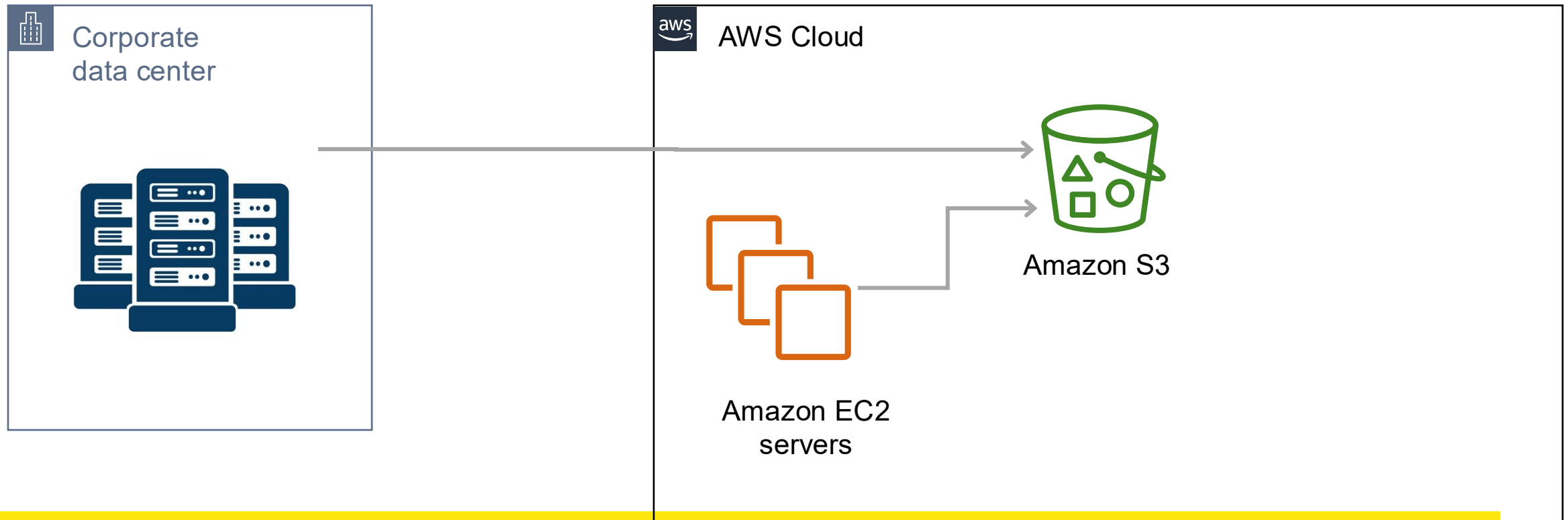


`https://<bucket-name>.s3.amazonaws.com/video.mp4`



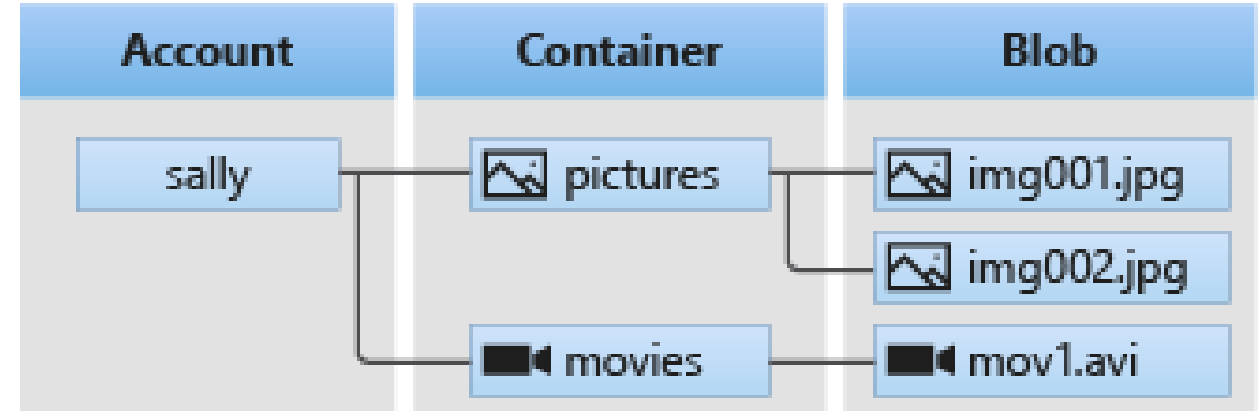
Object / Blob Storage – Use Cases

Object Storage as backup



Azure Storage Account

- Storage accounts are logical containers for multiple Azure storage services (blob, file, table, and queue).
- A storage account can contain N blob containers (similar to S3 buckets).
- The structure within the container is similar to folder structures (folders and blobs).



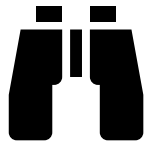
<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction>

Azure Storage Account



Durability

- Storage account durability between 11 nines and 16 nines, depending on SKU



Availability

- 99.9% availability SLA for hot tier (varies depending on SKU)



Scalability

- ~5 PB per Storage Account
- ~190 TB per Blob



Security

- Access via EntraID or access keys



Performance

- Support for various access patterns (block blob, page blob, append blob)

Azure Storage Account

- Storage account names must be globally unique.
- Containers must be unique at the same level in the storage account.

When naming your storage account, keep these rules in mind:

- Storage account names must be between 3 and 24 characters in length and may contain numbers and lowercase letters only.
- Your storage account name must be unique within Azure. No two storage accounts can have the same name.

`https://pclsdemo.blob.core.windows.net/assets/static/ag.svg`

Globally unique name

Container name
(2 Container)

Blob name

<https://learn.microsoft.com/en-us/azure/storage/common/storage-account-overview>

Azure Storage Account - Redundancy

Storage accounts are region-level resources. The region determines the data residency.

Redundancy	Description	Benefits
LRS Locally Redundant Storage	Redundant within the region. No control over AZ in the region.	Cost-effective 3 replicas of the data Writes are synchronous
ZRS Zone Redundant Storage	Replication of data across 3 or more zones in the region.	Fail-safe for AZ failures Writes are synchronous
GRS / Geo-Redundant Storage GZRS / Geo-Zone Redundant Storage	Replication of data to region pair. Distribution analogous to LRS/ZRS.	Fail-safe for regions Writes synchronously in primary region and asynchronously in secondary region

<https://learn.microsoft.com/en-us/azure/storage/common/storage-redundancy>

Azure Storage Account - Concurrency

- Azure Storage Account supports three concurrency models
- Standard: Last Writer Wins

Optimistic Concurrency

- The application must check whether the data has been changed during write operations.
- Applications ensure that write operations do not overwrite each other.
- Error handling for the user is required.

Pessimistic Concurrency

- The application creates a lock on the object to be modified
- Error handling may be necessary for objects with locks

Last Writer Wins

- Die Applikation schreibt direkt, ohne zu prüfen ob die Daten in der Zwischenzeit geändert wurden

<https://learn.microsoft.com/en-us/azure/storage/blobs/concurrency-manage#optimistic-concurrency>

Azure Storage Account – Lifecycle Policies

- Automated management of objects with increasing storage requirements
- Use cases
 - Archiving data after, for example, 90 days without access
 - Deleting data after, for example, 180 days
 - Deleting “old” versions of data, but retaining the “latest” version
- Example policy Deletes blobs after 365 days without modification

```
{
  "rules": [
    {
      "name": "expirationRule",
      "enabled": true,
      "type": "Lifecycle",
      "definition": {
        "filters": {
          "blobTypes": [ "blockBlob" ]
        },
        "actions": {
          "baseBlob": {
            "delete": { "daysAfterModificationGreaterThan": 365 }
          }
        }
      }
    }
  ]
}
```

<https://learn.microsoft.com/en-us/azure/storage/blobs/lifecycle-management-policy-delete>

Azure Storage Account – Access Tiering

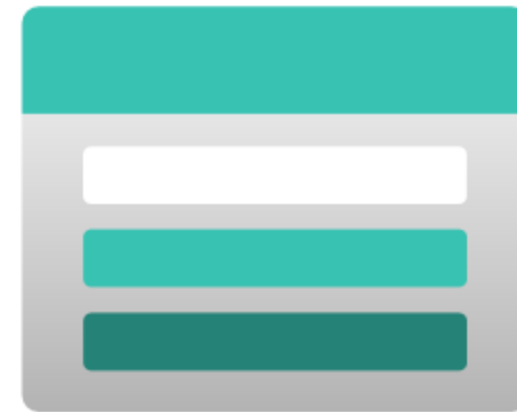
Access tiers serve to optimize costs, access speed, and retention depending on the use case.

Tier	Description	Cost Structure		Access Speed
Hot	High-frequency use of data	Cheap	Expensive	Immediately
Cool	Low-frequency use of data	Read/Write	Storage (GB)	Immediately
Cold	Rare use of data			Immediately
Archive	Archiving of data			Up to 15h to rehydrate
		Expensive	Cheap	

<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction>

Azure Storage Account - Emulator

- Storage Account equivalent solution for local development / offline development
- Storage Explorer: Legacy solution
- Azurite Emulator: Successor to the Storage Emulator, feature set not yet on par



<https://learn.microsoft.com/en-us/azure/storage/common/storage-use-azurite>

Azure Storage Account - Permissions

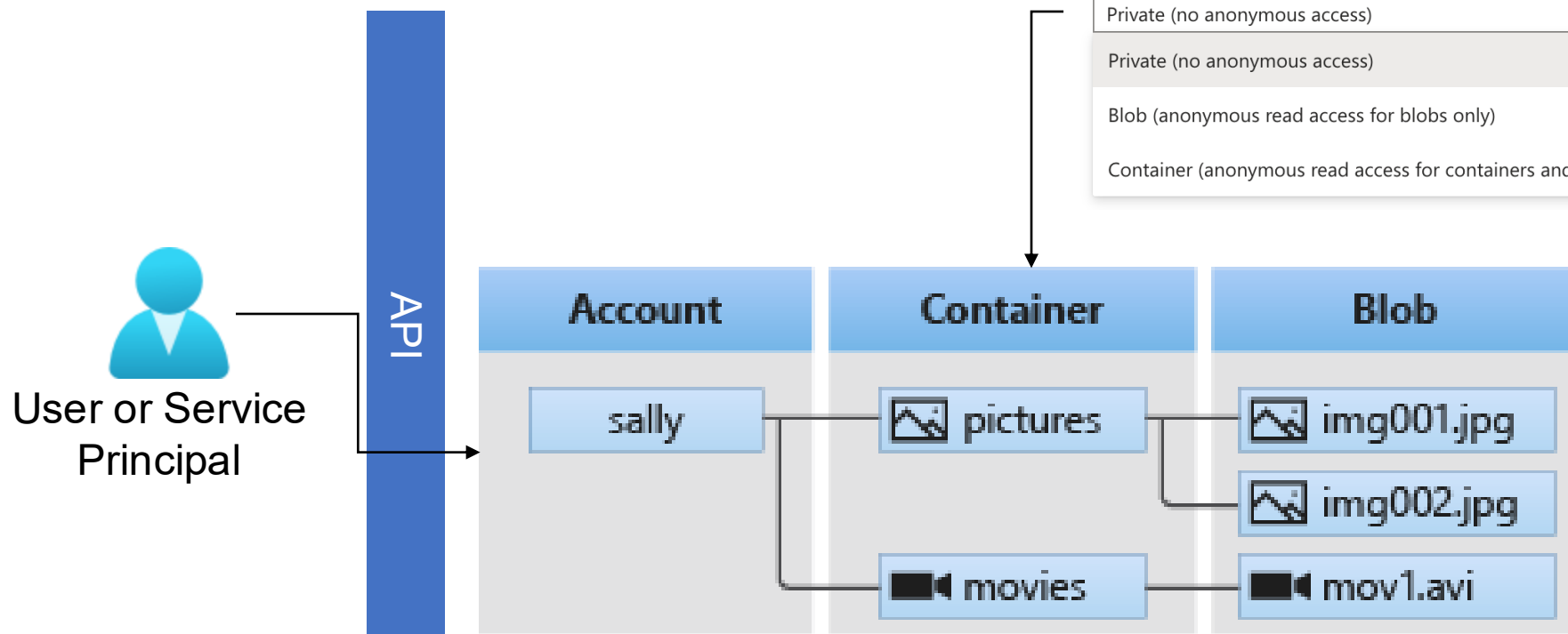
- Permission options vary depending on storage type (blob, file, queue, etc.)
- Distinction between access via
 - EntraID and Azure RBAC
 - Storage account access keys (or SAS keys derived from them)
 - Anonymous

Authorization option	Guidance	Recommendation
Microsoft Entra ID	Authorize access to Azure Storage data with Microsoft Entra ID	Microsoft recommends using Microsoft Entra ID with managed identities to authorize requests to blob resources.
Shared Key (storage account key)	Authorize with Shared Key	Microsoft recommends that you disallow Shared Key authorization for your storage accounts.
Shared access signature (SAS)	Using shared access signatures (SAS)	When SAS authorization is necessary, Microsoft recommends using user delegation SAS for limited delegated access to blob resources. SAS authorization is supported for Blob Storage and Data Lake Storage, and can be used for calls to blob endpoints and dfs endpoints.
Anonymous read access	Overview: Remediating anonymous read access for blob data	Microsoft recommends that you disable anonymous access for all of your storage accounts.
Storage Local Users	Supported for SFTP only. To learn more, see Authorize access to Blob Storage for an SFTP client	See guidance for options.
Microsoft Purview protection policies	Supported for Azure Blob Storage and Azure Data Lake Storage. To learn more, see Authoring and publishing protection policies for Azure sources (Preview) .	See guidance for options.

<https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access?tabs=blobs>

Azure Storage Account – Permissions for Blobs – Blob Access Level

- Access levels are defined per (top-level) container
- Anonymous access can be disabled for storage accounts



Change access level

Change the access level of container 'assets'.

Anonymous access level ⓘ

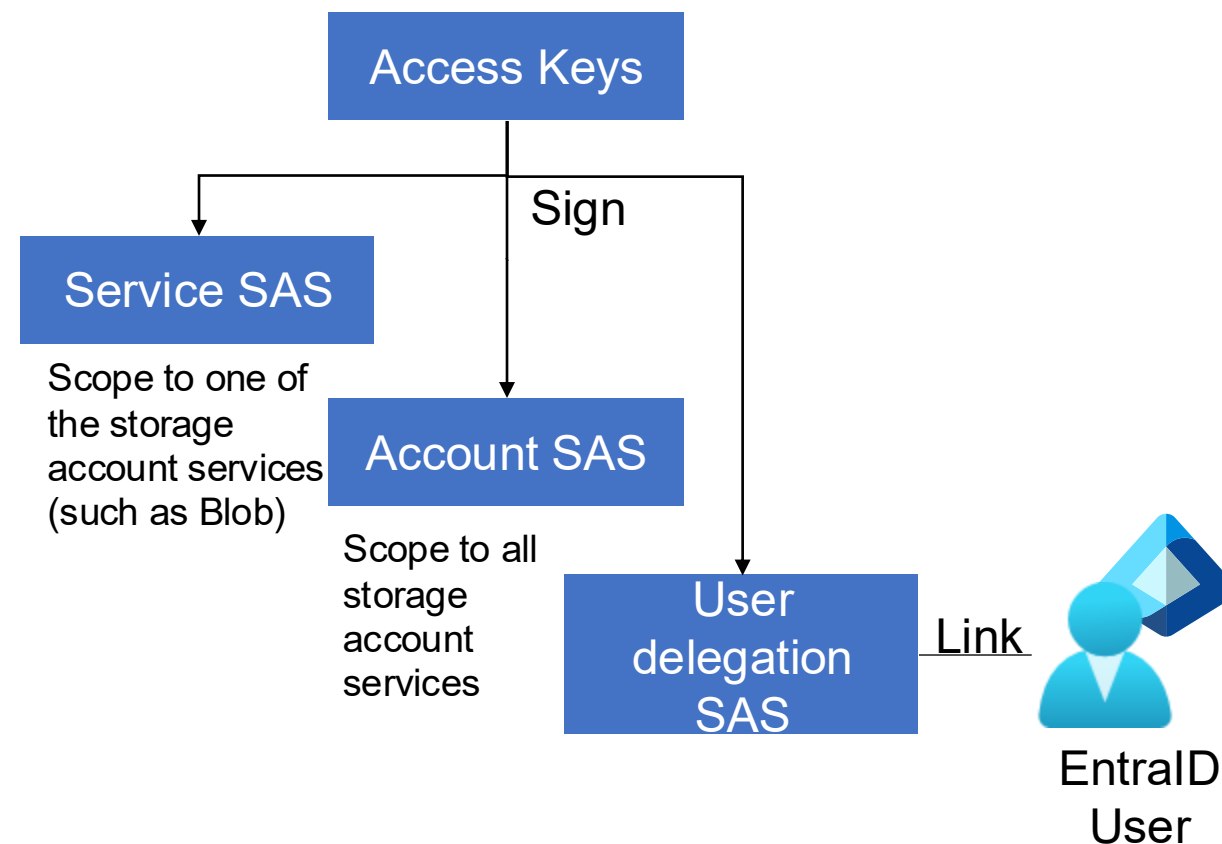
- Private (no anonymous access)
- Private (no anonymous access)
- Blob (anonymous read access for blobs only)
- Container (anonymous read access for containers and blobs)

```
curl https://pclsdemo.blob.core.windows.net/assets/static/ag.svg > ag.svg
```

<https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access?tabs=blobs>

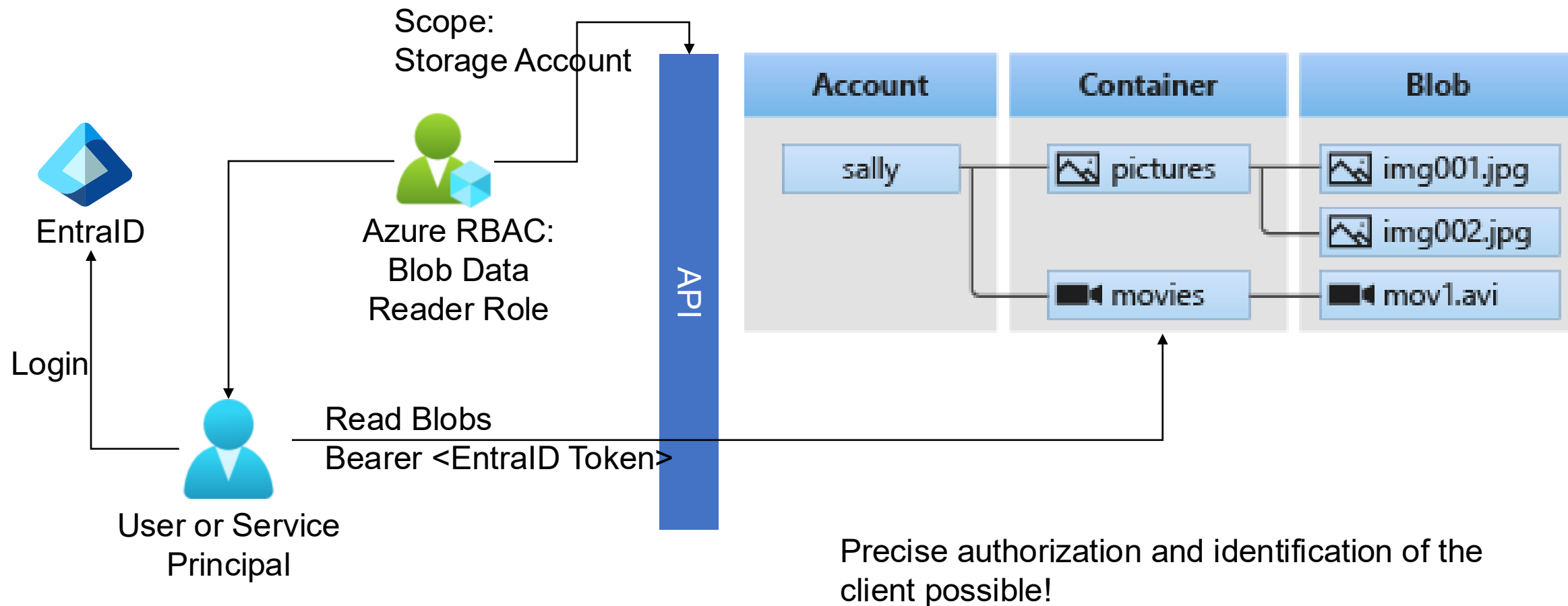
Azure Storage Account – Permissions for Blobs – Access Keys

- Access keys are administrative keys for all data and management operations.
- Access keys are used to sign SAS keys (shared access signature).
- The user/service is usually not identifiable.
- Access keys can be deactivated.
- SAS are used by clients to access data in blob storage.



<https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access?tabs=blobs>

Azure Storage Account – Permissions for Blobs - EntraID



<https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access?tabs=blobs>

Azure Storage Account – Dataimport

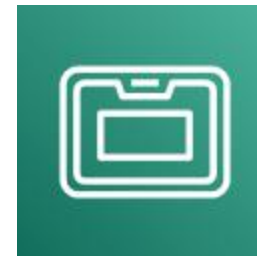
- Importing large amounts of data into Azure Storage Accounts / AWS S3
- Data volumes are in the petabyte (Snowball) range, starting at 40 terabytes with DataBox
- Transfer via the Internet / VPN not possible due to volume or bandwidth



Azure DataBox



Azure Import/Export



AWS
Snowball



AWS
Snowmobile

<https://learn.microsoft.com/de-de/azure/databox/data-box-overview?pivots=dbx-ng> /
<https://aws.amazon.com/de/snowball/>

Azure Storage Account – Dataimport

- Data import via network (VPN or Internet) into Azure Storage Accounts is performed using
 - AzCopy (CLI)
 - Azure Data Factory (data integration solution)
 - Blobfuse (POSIX wrapper for Blob API)
 - Azure Storage Data Movement Library for .NET
- Sources can be AWS S3 or existing file shares, for example.

Single files:

```
azcopy copy '<local-file-path>' 'https://<storage-account-name>.<blob or  
dfs>.core.windows.net/<container-name>/<blob-name>'
```

Folders:

```
azcopy copy '<local-directory-path>' 'https://<storage-account-name>.<blob or  
dfs>.core.windows.net/<container-name>' --recursive
```

<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction#move-data-to-blob-storage>

Azure Storage Account – Pricing

- Cost factors for blob storage
 - Data volume
 - Access frequency
 - Access tier
- Optimization through
 - Choice of the appropriate access tier
 - Use of lifecycle policies
 - Reserved capacity

Data storage prices pay-as-you-go	Premium	Hot	Cool	Archive
First 50 terabyte (TB)/month	CHF 0.17968 per GB	CHF 0.0176 per GB	CHF 0.00879 per GB	CHF 0.00159 per GB
Next 450 TB/month	CHF 0.17968 per GB	CHF 0.0169 per GB	CHF 0.00879 per GB	CHF 0.00159 per GB
Over 500 TB/month	CHF 0.17968 per GB	CHF 0.0162 per GB	CHF 0.00879 per GB	CHF 0.00159 per GB

Operations and data transfer	Premium	Hot	Cool	Archive
Write operations (per 10,000) ¹	CHF 0.0274	CHF 0.0561	CHF 0.1039	CHF 0.1371
Read operations (per 10,000) ¹	CHF 0.0022	CHF 0.0045	CHF 0.0104	CHF 6.8516
Archive High Priority Read (per 10,000) ¹				CHF 74.2253
Iterative Read Operations (per 10,000) ¹	CHF 0.0274	CHF 0.0045	CHF 0.0045	CHF 0.0045

<https://azure.microsoft.com/en-us/pricing/details/storage/blobs/>

Storage – Summary

- Block storage
 - Directly attached to VMs in the form of disks
 - Use only via VMs
- File storage
 - POSIX-compatible storage in the form of shares
 - Use by VMs and other services via network
- Blob storage
 - Object storage in the form of buckets or storage account containers
 - Use by any clients via REST API



Demo

- Create storage account with IaC
- Upload/download blob via keys

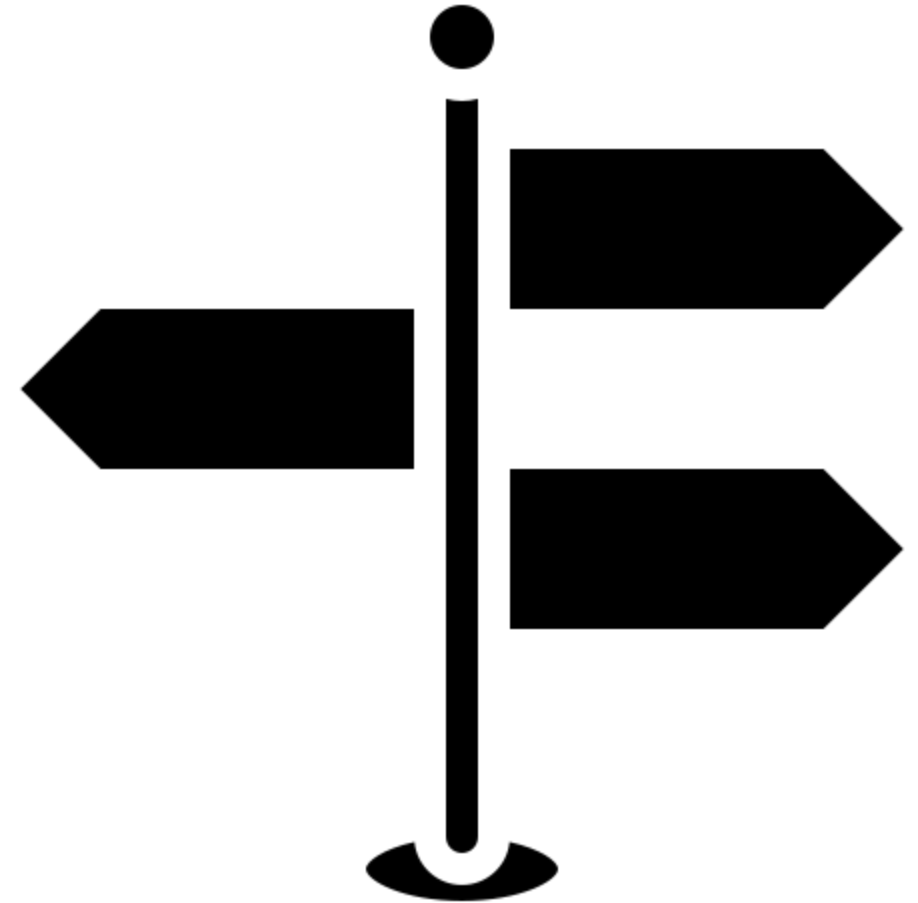
Exercise

- Create your own storage account via the portal.
- Make blobs accessible on a time-based basis via shared access signature (i.e. access expiration after a few mins)
- Make blobs publicly accessible with anonymous access.

<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction>

Learning Objectives

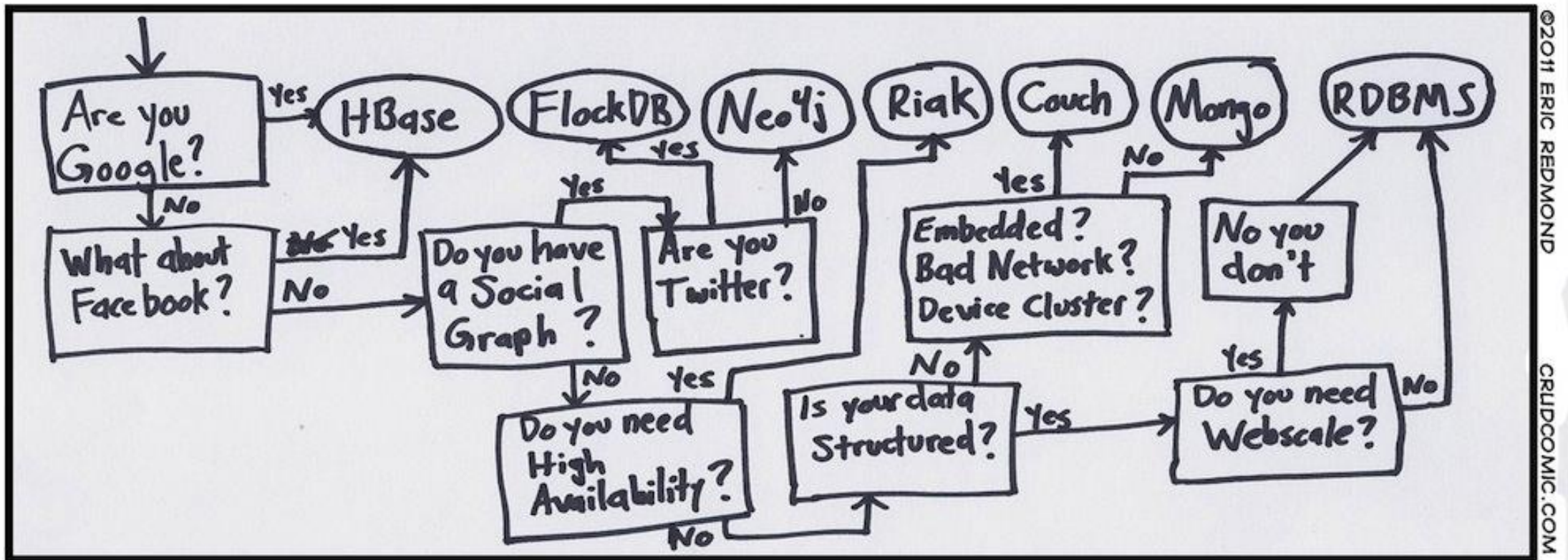
- Students gain an overview of DBaaS (Database as a Service) and understand the challenges of operating databases in the cloud.
- Students can differentiate between different types of DBaaS and use them correctly in context.
- Students understand DBaaS licensing models.



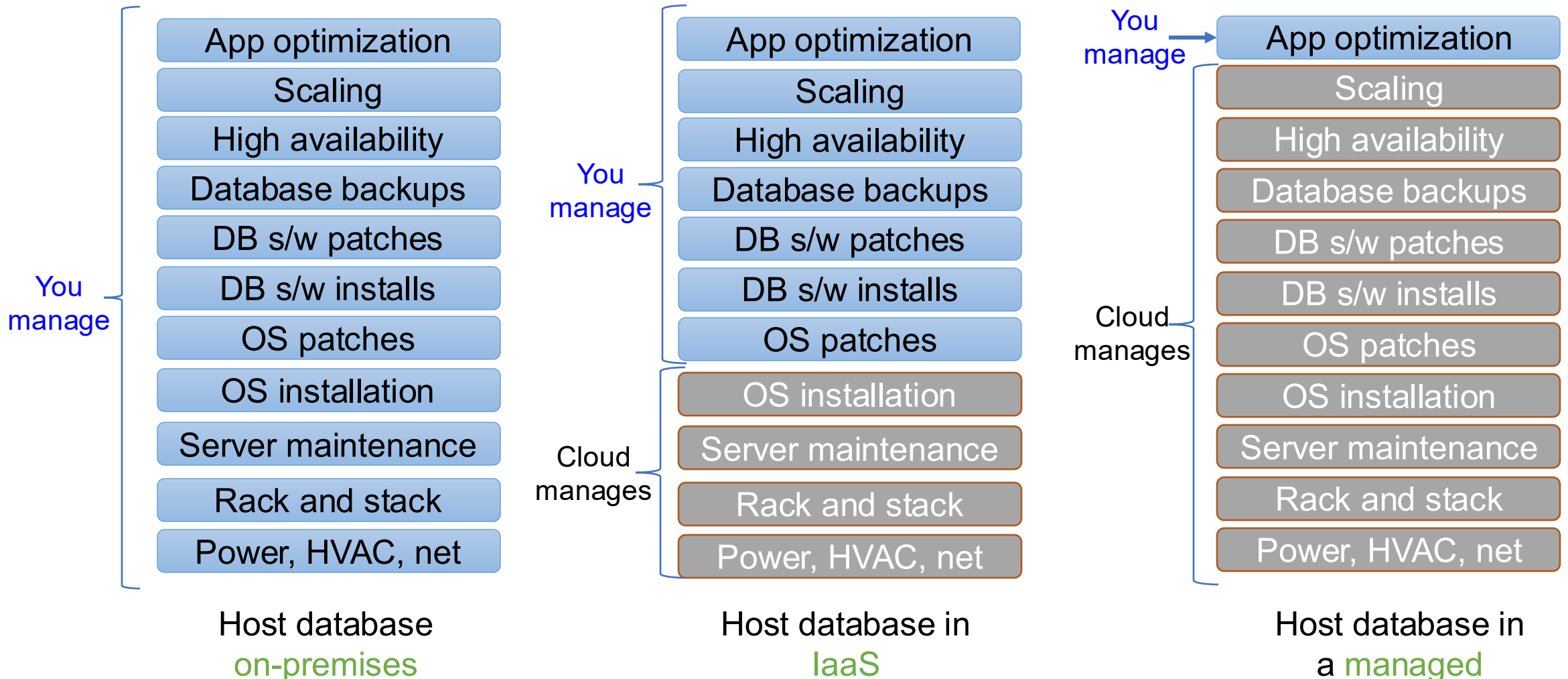
Agenda

- CAP
- DB Services
- Example: Cosmos and Postgres
- Integration into app
- IaC

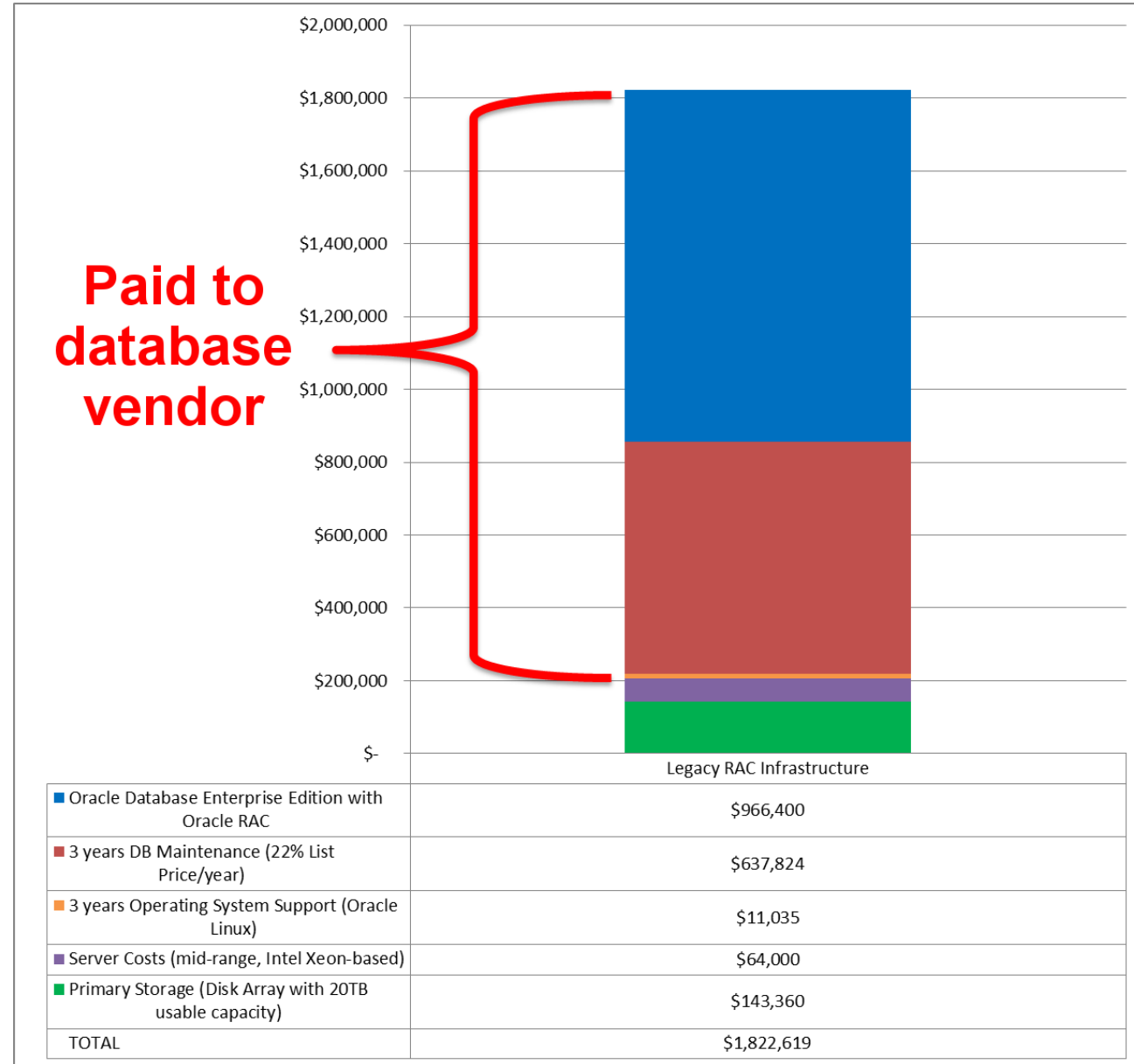
Motivation



Why Cloud Databases



Why Cloud Databases



CAP-Theorem

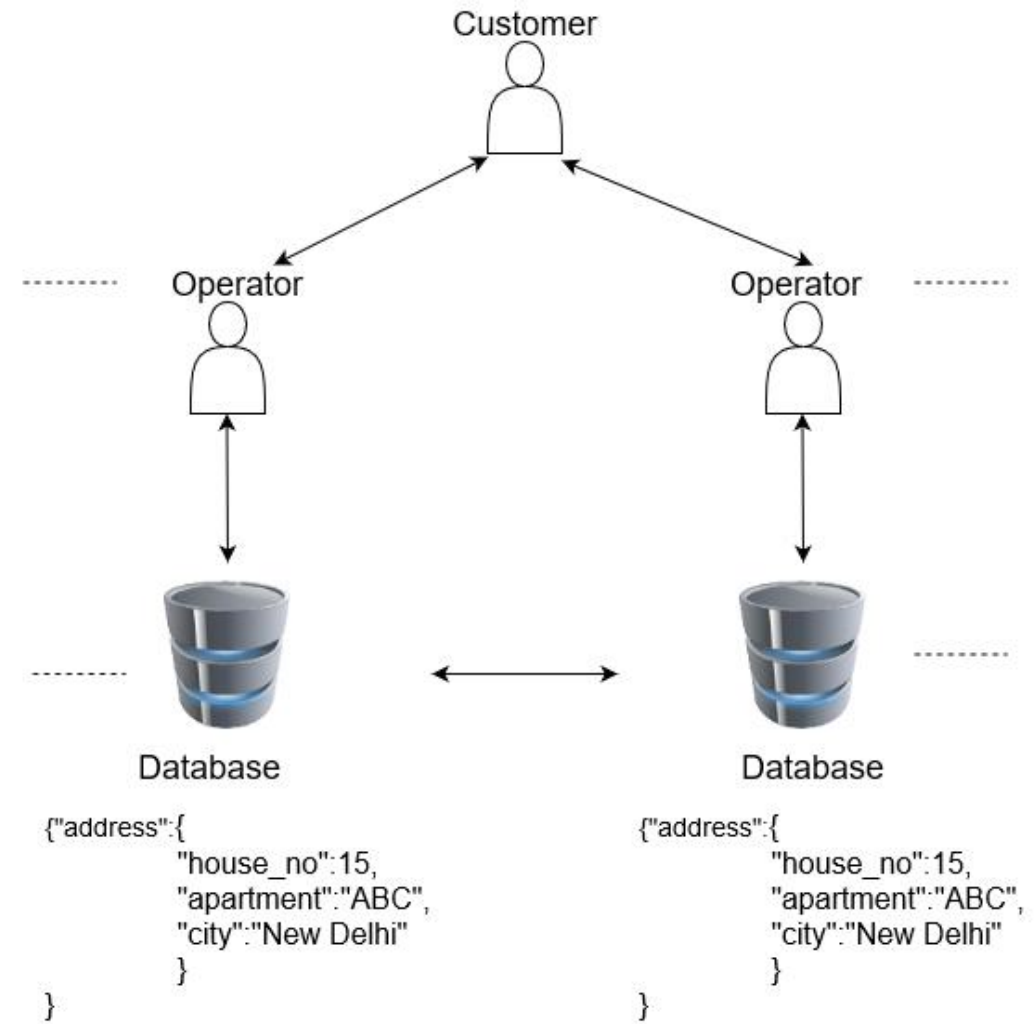
“It is impossible for a web service to provide these *three guarantees at the same time (pick 2 of 3)*:

- **(Sequential) Consistency**
- **Availability**
- **Partition-tolerance”**

Eric Brewer: https://en.wikipedia.org/wiki/CAP_theorem

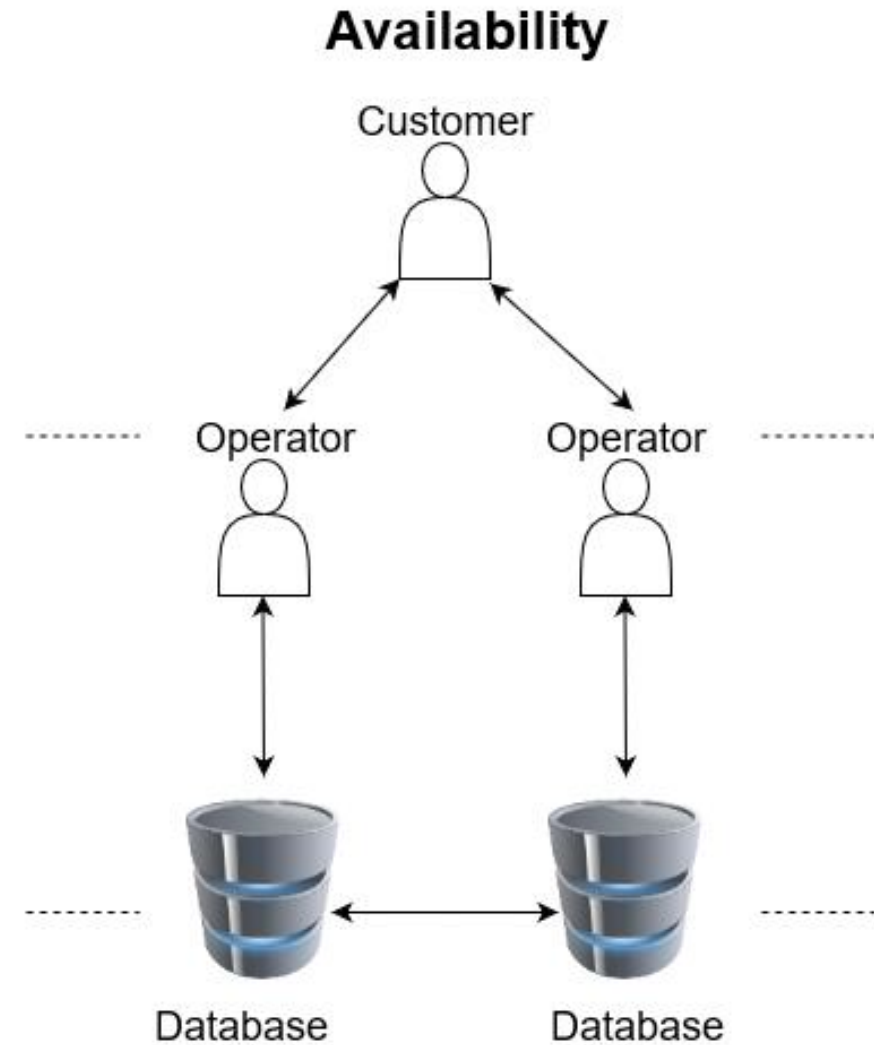
Consistency

- All nodes return the same data at the same time
- For successful write operations, all read operations subsequently respond to the new state



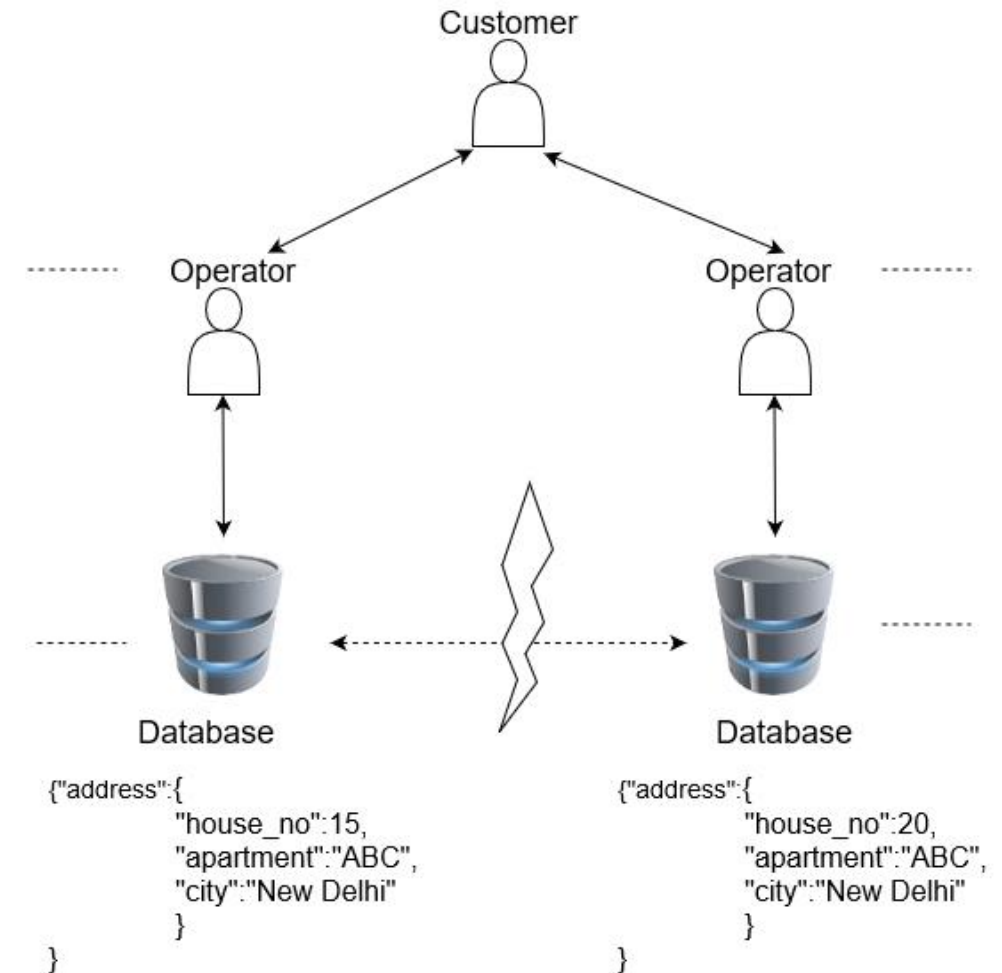
Availability

- Each functioning node can respond to requests without waiting for other nodes...
- also applies to write accesses
- Requests may receive outdated data as a response



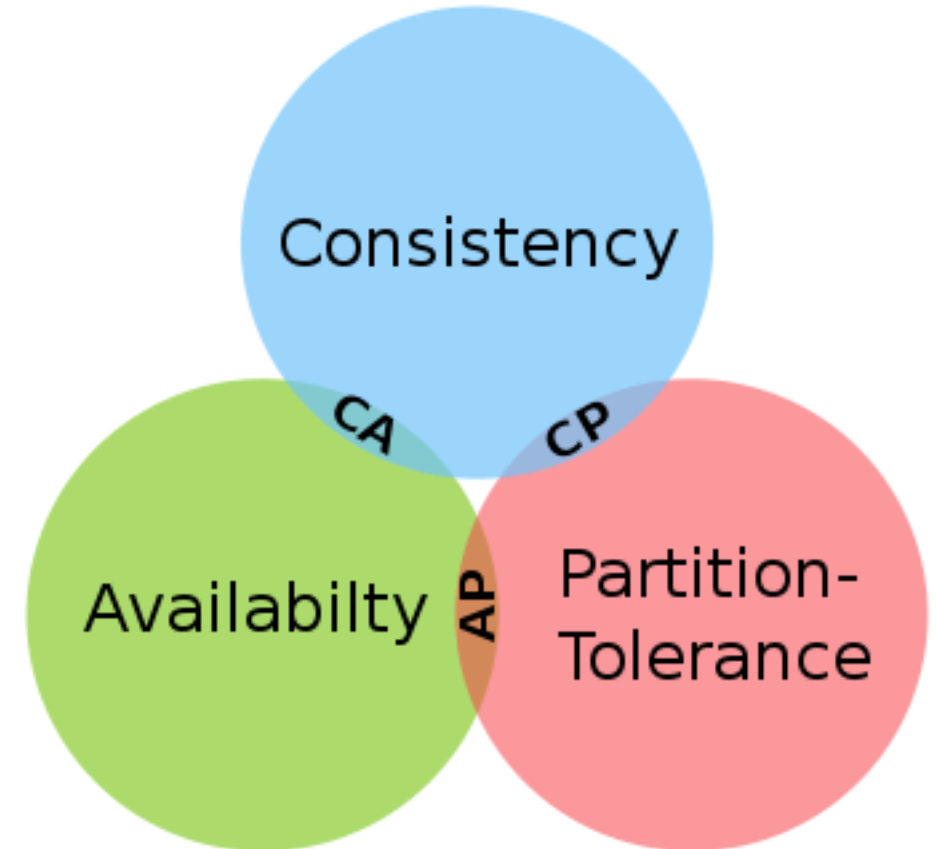
Partition Tolerance

- If the nodes can no longer communicate with each other, the individual nodes must still function normally on their own.
- P is unavoidable in distributed systems:
 - “Of the CAP theorem's Consistency, Availability, and Partition Tolerance, Partition Tolerance is mandatory in distributed systems. You cannot not choose it.” – Coda Hale
 - “An important observation is that in larger distributed-scale systems, network partitions are a given; therefore, consistency and availability cannot be achieved at the same time” – Werner Vogels



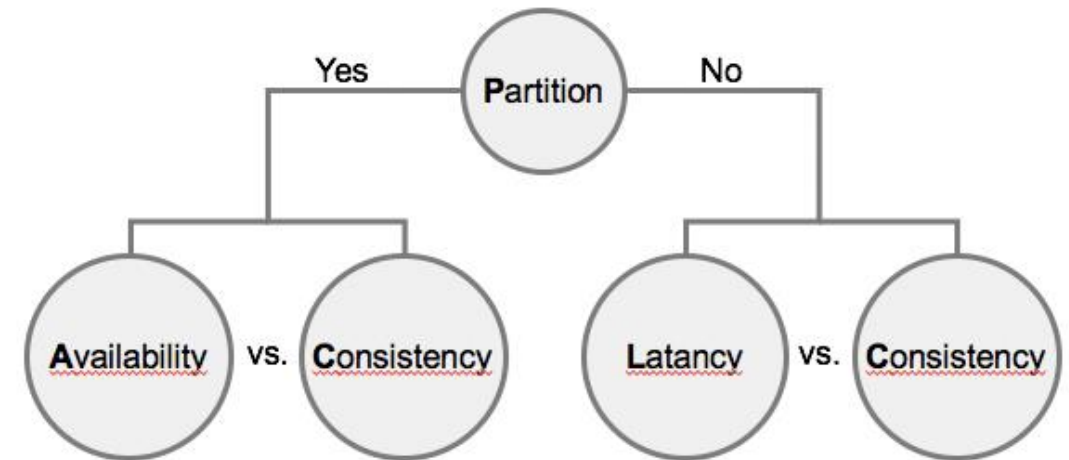
Implications on distributed systems

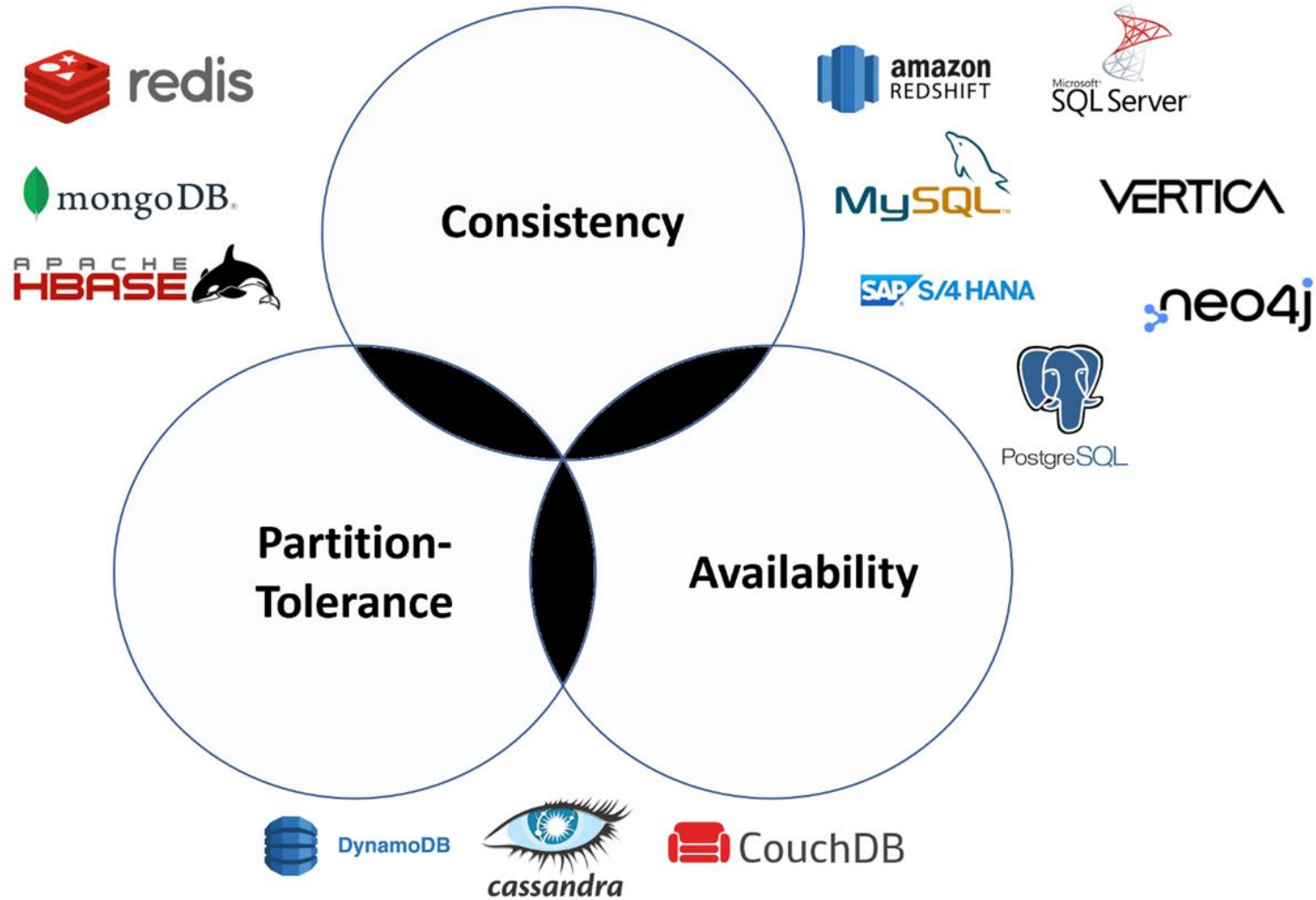
- Of the following guarantees in distributed systems:
 - Consistency
 - Availability
 - Partition tolerance
- You can choose a maximum of two at the same time.
- This results in three types of distributed systems for storing data:
 - CP
 - AP
 - CA



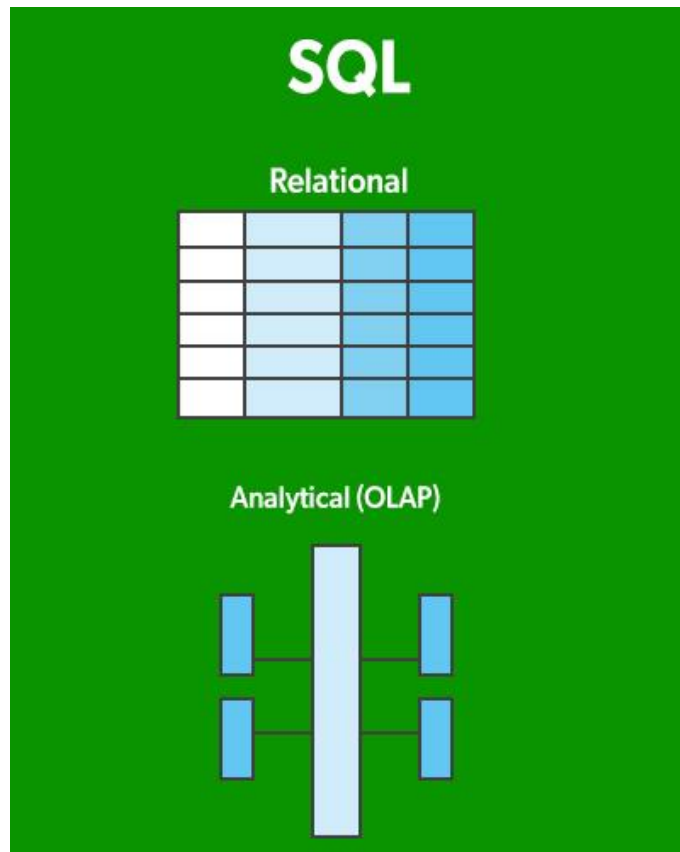
Summary CAP

- CAP should stimulate thinking: What is important in a “distributed” architecture?
- The devil is in the details: Complete architectures cannot be “simply” classified into CA/AP/CP.
- Databases, however, tend to be...
- Above all, it should be clear how a system behaves in a network partitioning case.
 - If there is no partitioning, everything should be consistent and available anyway.
 - Many data stores can still fall back on CAP.
- There is another extension: PACELC.



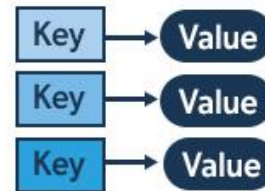


Database Classification

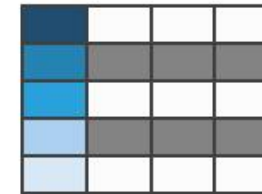


NoSQL

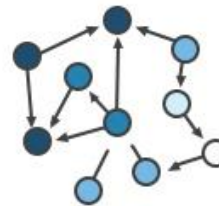
Key-Value



Column-Family



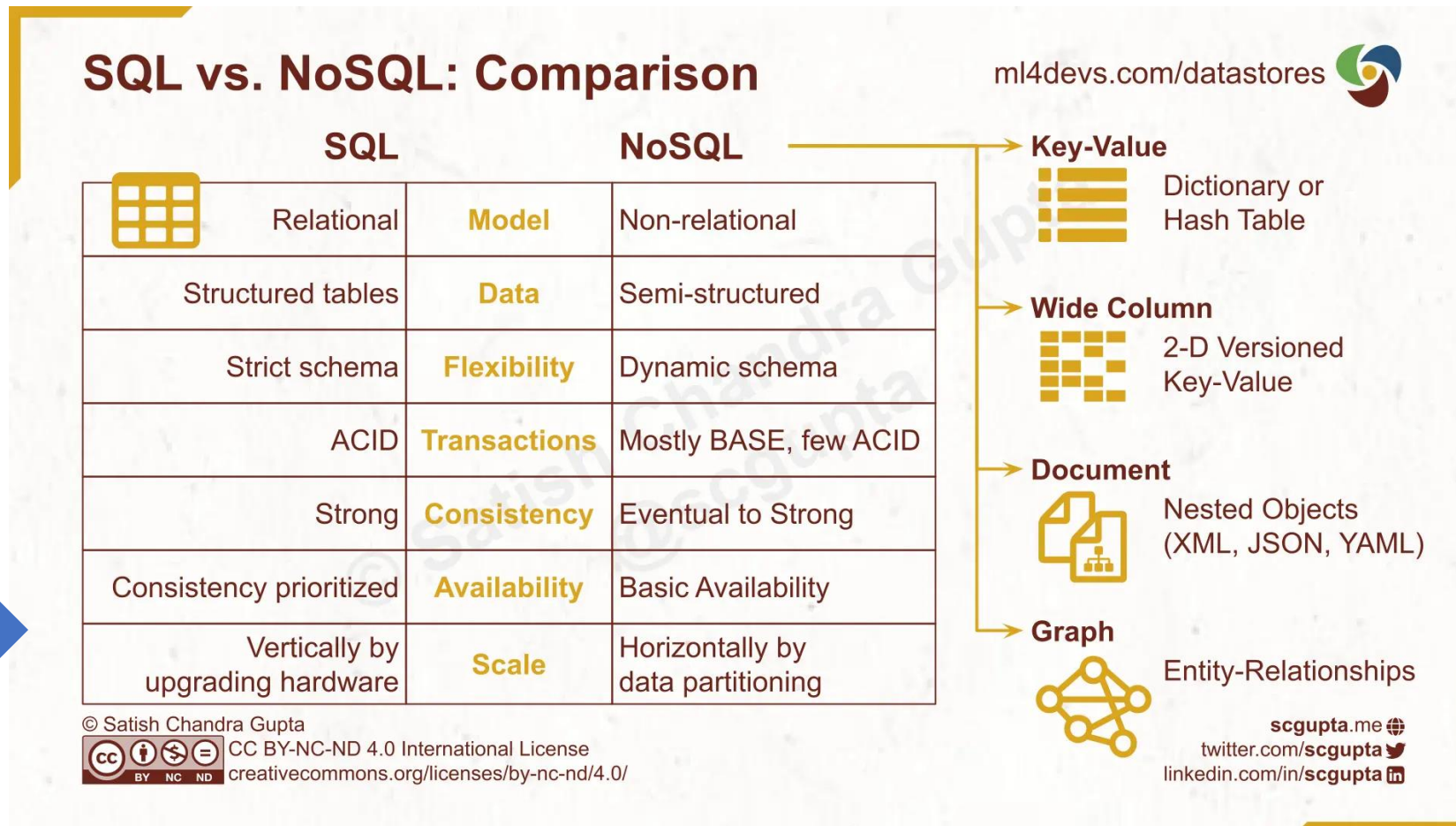
Graph



Document



Database Classification



ACID vs BASE

- Acronyms describing database behavior and characteristics
- Most SQL databases implement ACID, and most No-SQL databases implement BASE. However, there are exceptions!
- ACID: Atomicity, consistency, isolation, and durability.
- BASE: Basically available, soft state, and eventually consistent.



- STRONG CONSISTENCY
- ISOLATION
- TRANSACTIONS
- SCALE-UP (LIMITED)
- PRECISE ANSWERS
- CONSISTENCY FIRST



- WEAK CONSISTENCY
- LAST WRITE WINS
- DEVELOPER MANAGED
- SCALE-OUT (UNLIMITED)
- APPROXIMATE ANSWERS
- AVAILABILITY FIRST

@luminousmen.com

Relational Databases

Use is advisable when

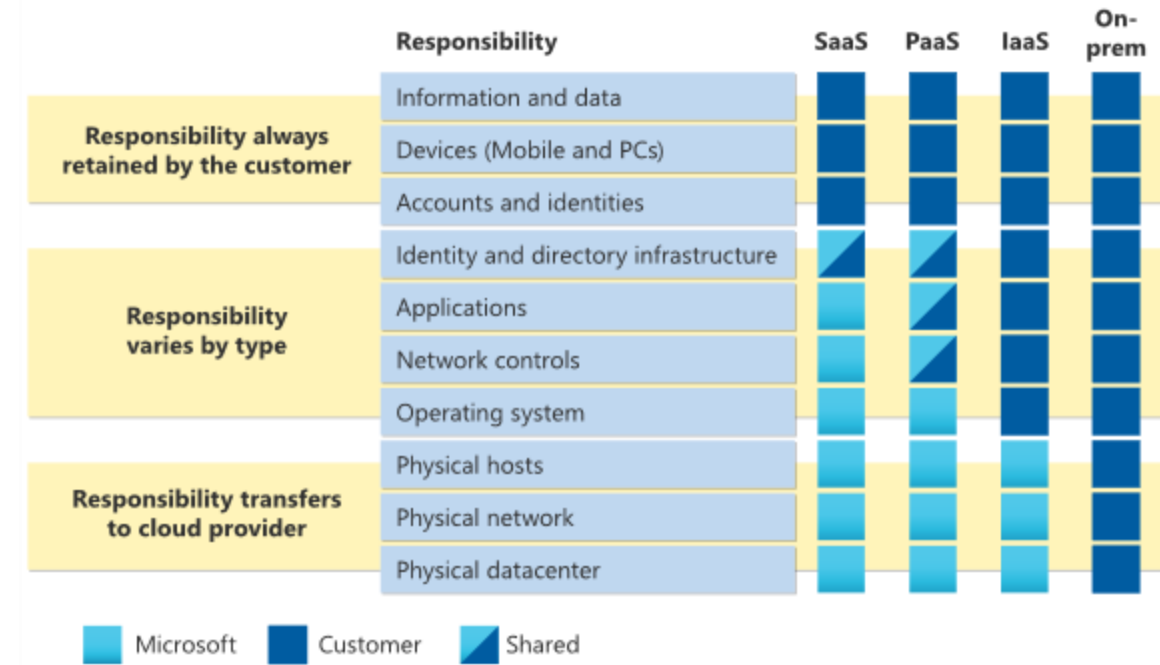
- Data schema is specified and must enforce rules (data quality)
- ACID necessary
- Number of read and write operations is limited
- Moderate performance necessary

Benefits

- User-friendliness
- Data integrity
- Low storage requirements
- SQL – Structured Query Language

Hosting on Azure

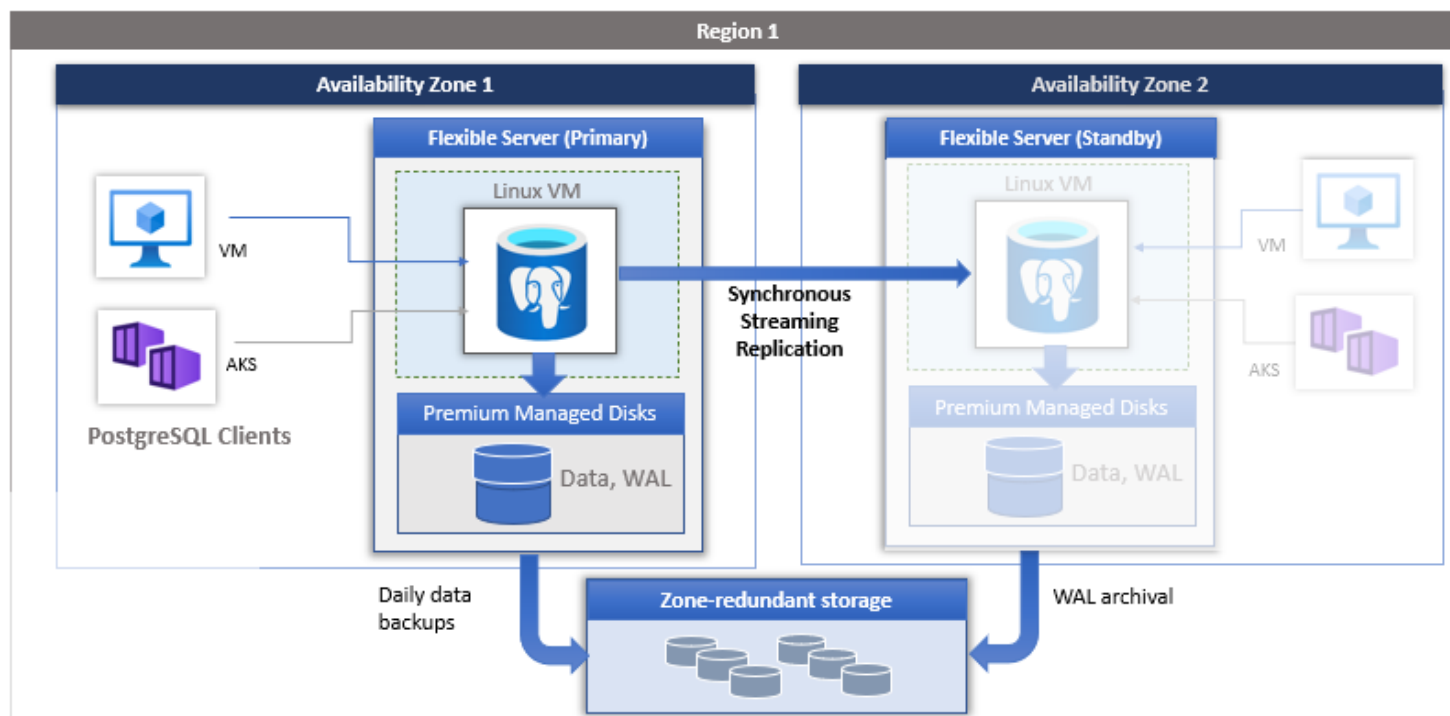
- IaaS: Self-managed IaaS VMs with, for example, SQL Server on top.
 - Customer bears overall responsibility.
- DBaaS: Managed database servers, e.g., Managed PostgreSQL.
 - Dedicated VMs for the instances.
 - Operation by the provider.
 - Personal responsibility for, e.g., backup and restore processes, data residency, and availability levels.



<https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>

Azure Database for PostgreSQL Flexible Server

- Managed PostgreSQL instance from Azure
- Configuration of
 - Data residency (region)
 - Replication (read replicas)
 - Backup intervals
 - Encryption
 - Network integration (private endpoint or similar)



Azure Database for PostgreSQL Flexible Server SKUs

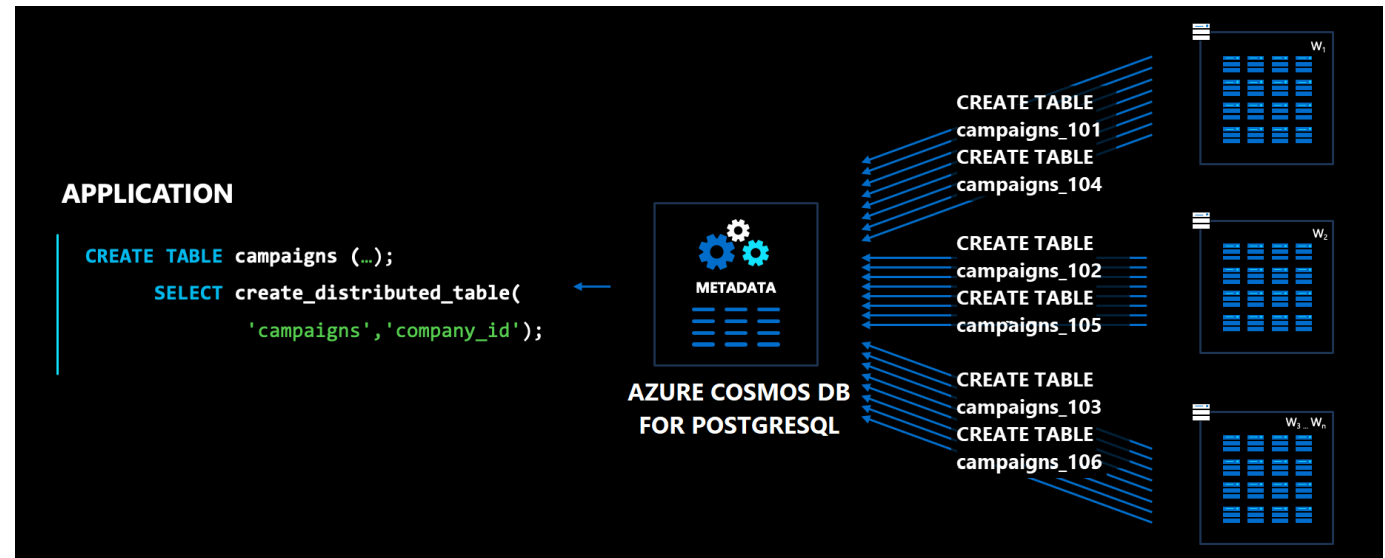
Resource/Tier	Burstable	General Purpose	Memory Optimized
VM-series	B-series	Ddsv5-series, Dadsv5-series, Ddsv4-series, Dsv3-series	Edsv5-series, Eadsv5-series, Edsv4-series, Esv3-series
vCores	1, 2, 4, 8, 12, 16, 20	2, 4, 8, 16, 32, 48, 64, 96	2, 4, 8, 16, 20 (v4/v5), 32, 48, 64, 96
Memory per vCore	Variable	4 GiB	6.75 GiB to 8 GiB
Storage size	32 GiB to 64 TiB	32 GiB to 64 TiB	32 GiB to 64 TiB
Automated Database backup retention period	7 to 35 days	7 to 35 days	7 to 35 days
Long term Database backup retention period	up to 10 years	up to 10 years	up to 10 years

<https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-compute>

Azure Cosmos DB for PostgreSQL

Azure Cosmos DB is a

- Fully managed DB service with various DB APIs (e.g., PostgreSQL, MongoDB, Apache Cassandra, and more)
- Optional configuration as a multi-region and multi-master service instance
- Features of specific APIs may not always correspond fully to the product (e.g., in the case of MongoDB)
- Hint: Cosmos DB for PostgreSQL is followed by newer services



NoSQL Datenbanken

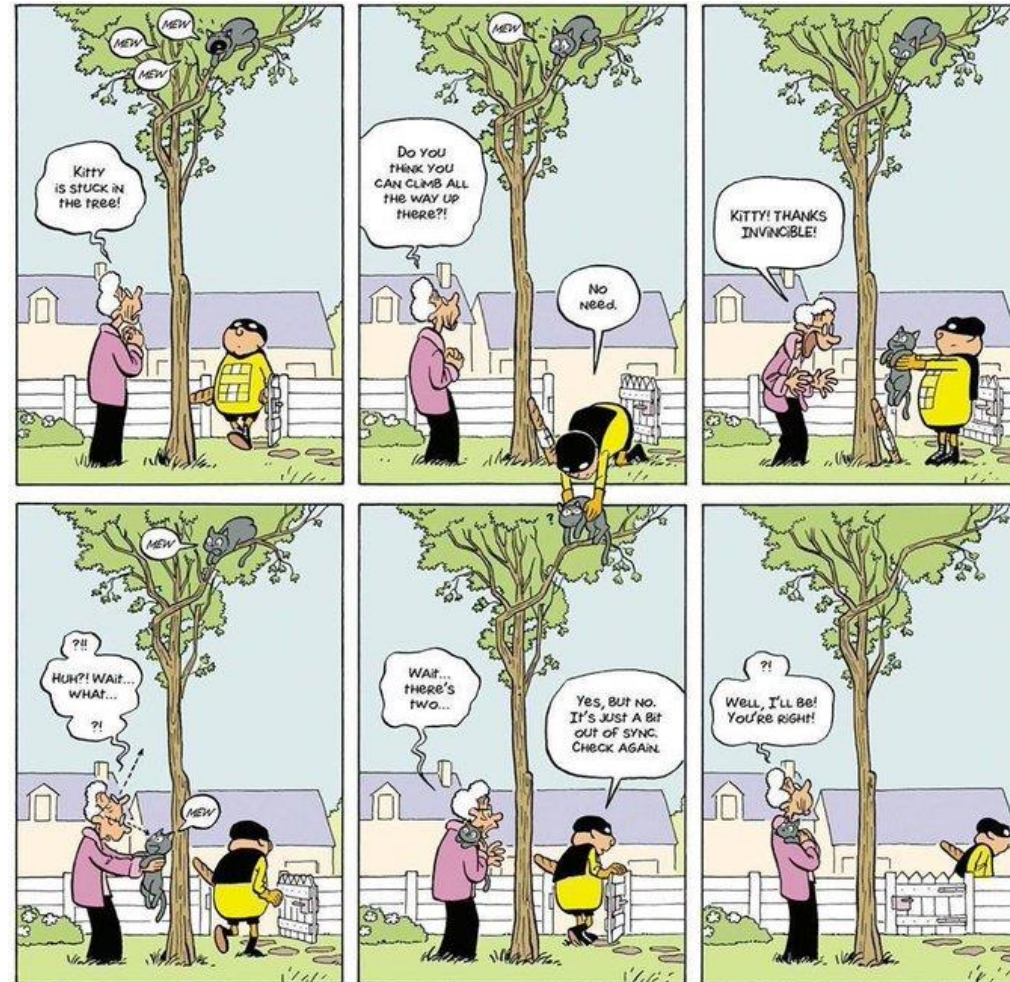
Use is advisable when

- Processing of high data volumes
- Horizontal scalability required (e.g., across regions)
- No fixed schemas for the data
- High volume of read and write operations

Benefits

- Flexibility
- Scalability
- High performance
- APIs

Eventual Consistency



Azure Cosmos DB Eventual Consistency



- Cosmos DB offers 5 consistency levels
- From left to right
 - Data must first be committed in all replicas vs. new data record is replicated step by step (e.g., across replicas and regions)
 - The latest version of the data record is read vs. an outdated state may be read

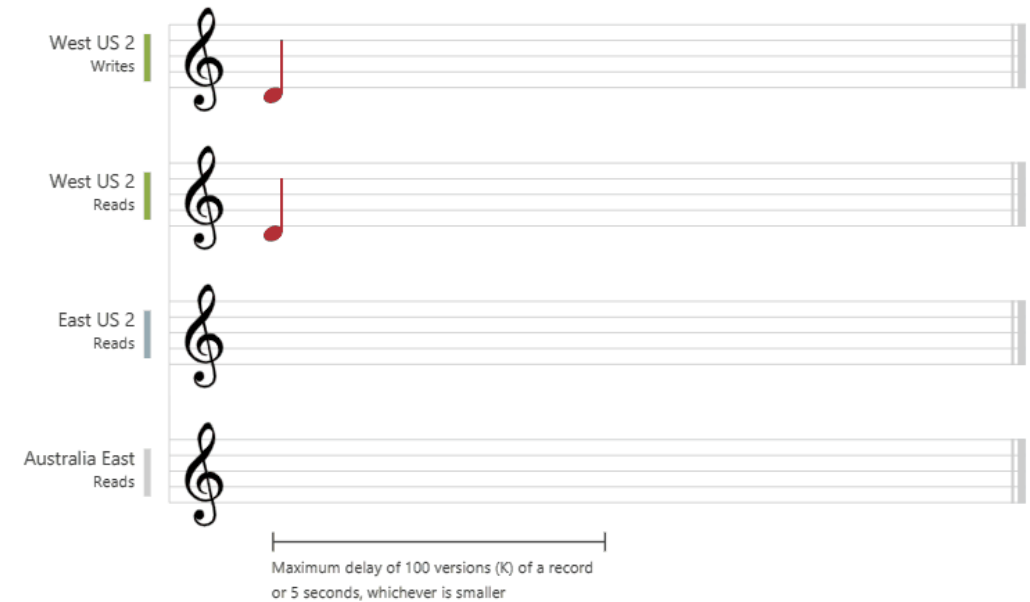
<https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>

Azure Cosmos DB Eventual Consistency

Strong consistency



Bounded staleness



<https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>

Azure Cosmos DB Eventual Consistency

Session consistency



Consistent prefix consistency



<https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>

Azure Cosmos DB Eventual Consistency

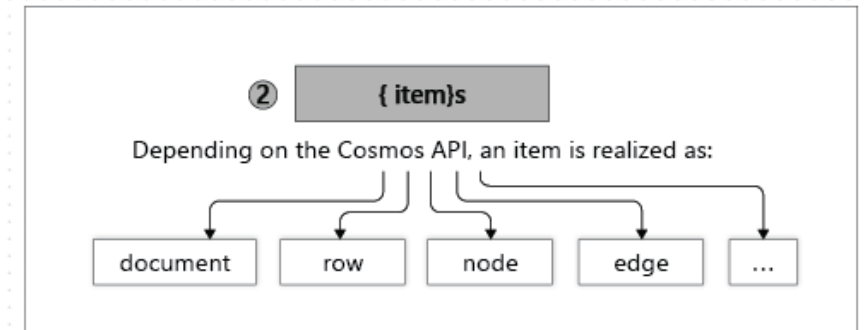
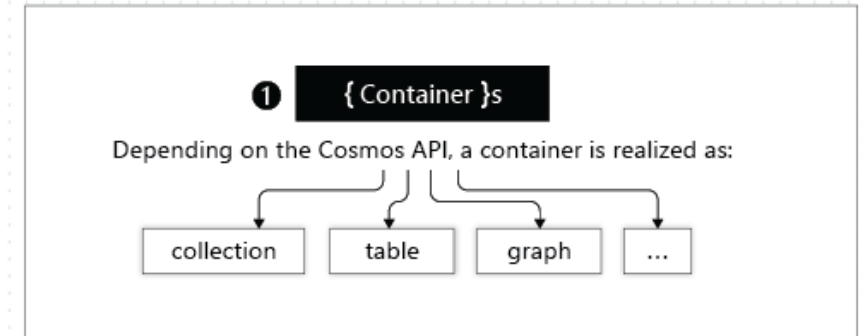
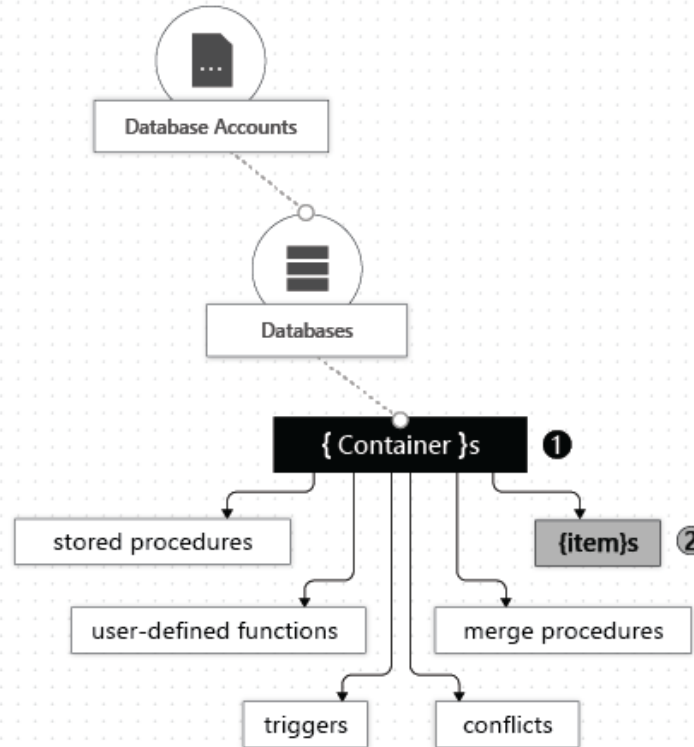


- Eventual consistency is useful for (globally) distributed applications with high performance and availability requirements.
- The consistency level must match the use case (outdated data may be displayed or needs to be handled by the app).
- Consistency levels can be selected for each API. Different defaults are used.

<https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>

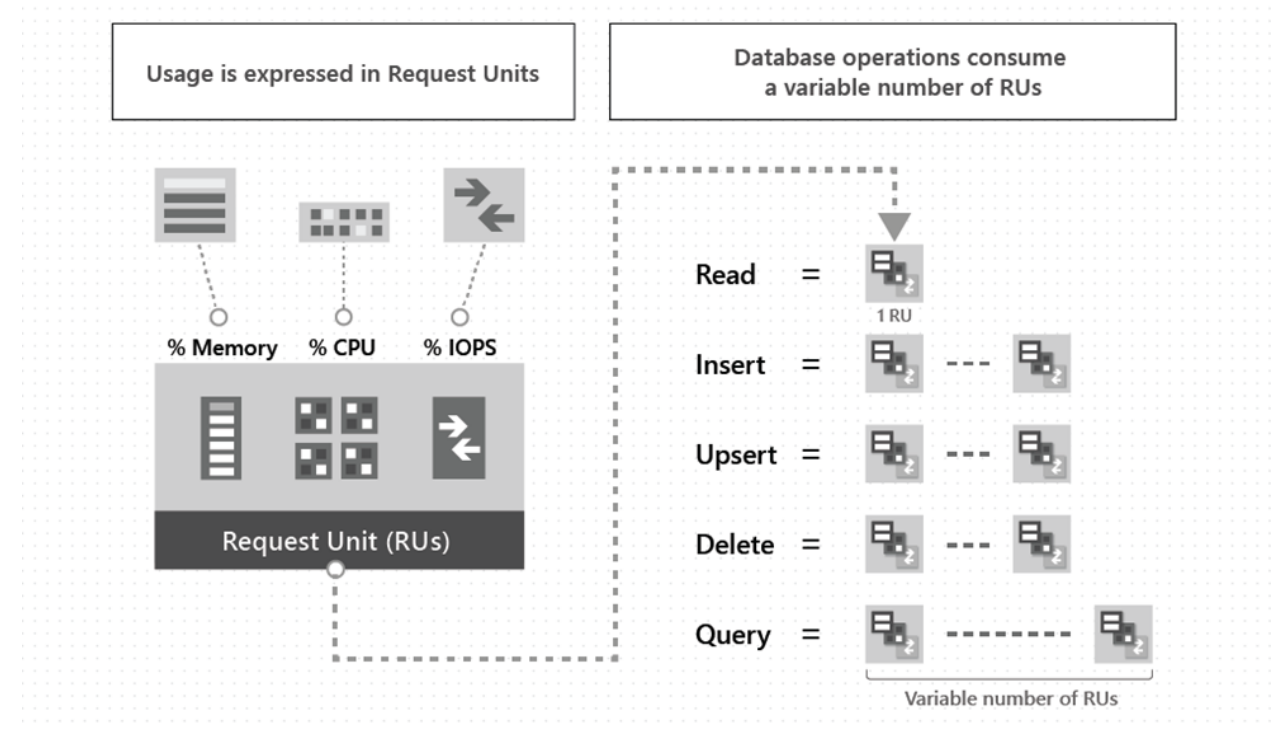
Cosmos DB No-SQL

- The generic model of Cosmos DB is mapped to the specific APIs
- Example DocumentDB:
 - Container = Collection
 - Item = Document



Cosmos DB - Pricing

- Compute Pricing
 - Request Units (RUs) – a generic unit for mapping CPU and memory
 - Alternatively, vCores
- Storage Pricing
 - Used storage (GB)
 - Alternatively, provisioned storage (disk size per Cosmos DB node)
- Resources are obtained via
 - Serverless SKU (effective consumption)
 - Provisioned throughput/storage (fixed amount of RUs or vCores)



<https://learn.microsoft.com/en-us/azure/cosmos-db/request-units>

Cosmos DB – Postgres API

- Cosmos DB instance with Postgres API compatibility
- vCore based deployment
- Coordinator: Contact point for clients, forwards queries to nodes

<https://learn.microsoft.com/en-us/azure/cosmos-db/postgresql/quickstart-create-bicep?tabs=CLI>

```
@secure()
param administratorLoginPassword string
param location string
param clusterName string
param coordinatorVCores int = 4
param coordinatorStorageQuotaInMb int = 262144
param coordinatorServerEdition string = 'GeneralPurpose'
param enableShardsOnCoordinator bool = true
param nodeServerEdition string = 'MemoryOptimized'
param nodeVCores int = 4
param nodeStorageQuotaInMb int = 524288
param nodeCount int
param enableHa bool
param coordinatorEnablePublicIpAddress bool = true
param nodeEnablePublicIpAddress bool = true
param availabilityZone string = '1'
param postgresqlVersion string = '15'
param citusVersion string = '12.0'

resource serverName_resource 'Microsoft.DBforPostgreSQL/serverGroupsV2@2022-11-08'
  name: clusterName
  location: location
  tags: {}
  properties: {
    administratorLoginPassword: administratorLoginPassword
    coordinatorServerEdition: coordinatorServerEdition
    coordinatorVCores: coordinatorVCores
    coordinatorStorageQuotaInMb: coordinatorStorageQuotaInMb
    enableShardsOnCoordinator: enableShardsOnCoordinator
    nodeCount: nodeCount
    nodeServerEdition: nodeServerEdition
    nodeVCores: nodeVCores
    nodeStorageQuotaInMb: nodeStorageQuotaInMb
    enableHa: enableHa
    coordinatorEnablePublicIpAddress: coordinatorEnablePublicIpAddress
    nodeEnablePublicIpAddress: nodeEnablePublicIpAddress
    citusVersion: citusVersion
    postgresqlVersion: postgresqlVersion
    preferredPrimaryZone: availabilityZone
  }
}
```


Cosmos DB – SQL API

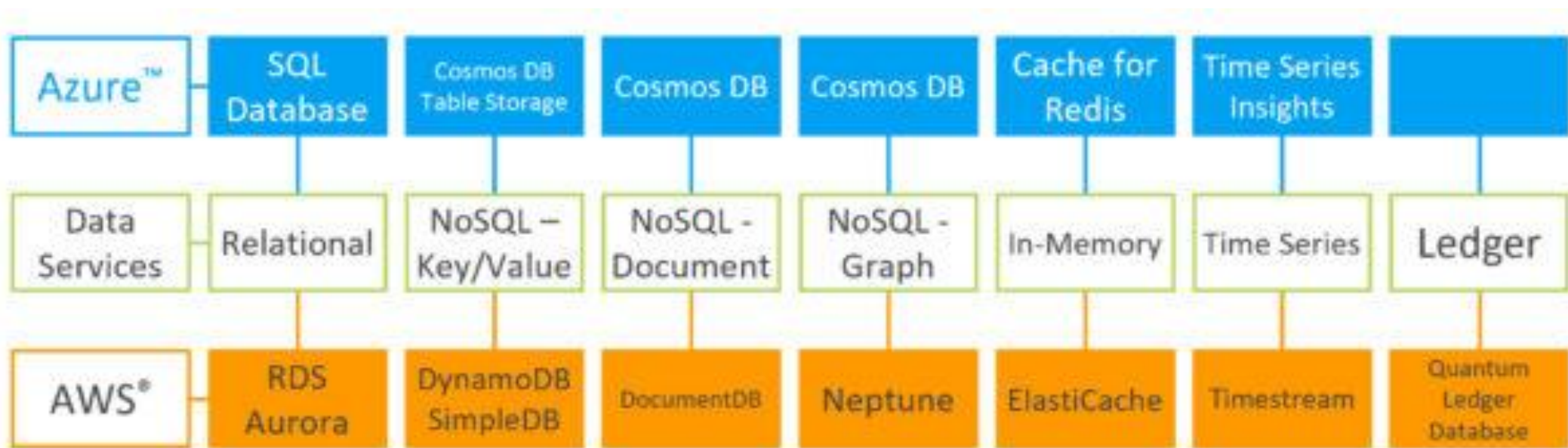
```
resource account 'Microsoft.DocumentDB/databaseAccounts@2023-11-15' = {
  name: toLower(accountName)
  location: location
  properties: {
    enableFreeTier: true
    databaseAccountOfferType: 'Standard'
    consistencyPolicy: {
      defaultConsistencyLevel: 'Session'
    }
    locations: [
      {
        locationName: location
      }
    ]
  }
}

resource database 'Microsoft.DocumentDB/databaseAccounts/sqlDatabases@2023-11-15'
parent: account
name: databaseName
properties: {
  resource: {
    id: databaseName
  }
  options: {
    throughput: 1000
  }
}
```

```
resource container 'Microsoft.DocumentDB/databaseAccounts/sqlDatabases/containers@2023-11-15'
parent: database
name: containerName
properties: {
  resource: {
    id: containerName
    partitionKey: {
      paths: [
        '/myPartitionKey'
      ]
      kind: 'Hash'
    }
  }
  indexingPolicy: {
    indexingMode: 'consistent'
    includedPaths: [
      {
        path: '/*'
      }
    ]
    excludedPaths: [
      {
        path: '/_etag/?'
      }
    ]
  }
}
```

<https://learn.microsoft.com/en-us/azure/cosmos-db/nosql/manage-with-bicep#free-tier-azure-cosmos-db-account>

Summary DB Services



Demo: CosmosDB with Functions

Databases - Summary

- Relational databases are primarily based on ACID, whereas most No-SQL databases are based on BASE.
- Database as a Service transfers OS and database maintenance tasks to the cloud provider.
- Cost savings in database operation are offset via OPEX.
- Existing database technologies and versions may vary depending on the region and cloud provider.



Hausaufgaben

- Review the material from this lesson.
- Complete [Assignment 09](#).
- Create and deploy your own AI service instances with the following learning paths (if needed):
 - [Get started with Azure Cosmos DB for NoSQL](#)
 - [Azure Database for PostgreSQL Tutorials](#)

